

A Survey on the Optimization of Large Language Model-based Agents

SHANGHENG DU, Shanghai Institute of Artificial Intelligence for Education, East China Normal University, Shanghai, China and School of Computer Science and Technology, East China Normal University, Shanghai, China

JIABAO ZHAO, School of Computer Science and Technology, Donghua University, Shanghai, China

JINXIN SHI, Shanghai Institute of Artificial Intelligence for Education, East China Normal University, Shanghai, China and School of Computer Science and Technology, East China Normal University, Shanghai, China

ZHENTAO XIE, School of Computer Science and Technology, East China Normal University, Shanghai, China

XIN JIANG, School of Computer Science and Technology, East China Normal University, Shanghai, China

YANHONG BAI, School of Computer Science and Technology, East China Normal University, Shanghai, China

LIANG HE, School of Computer Science and Technology, East China Normal University, Shanghai, China

With the rapid development of Large Language Models (LLMs), LLM-based agents have been widely adopted in various fields, becoming essential for autonomous decision-making and interactive tasks. However, current work typically relies on prompt design or fine-tuning strategies applied to vanilla LLMs, which often leads to limited effectiveness in complex agent-related environments. Although numerous recent studies have explored various strategies to optimize LLM-based agents for complex agent tasks, a systematic review summarizing and comparing these methods from a holistic perspective remains lacking. In this survey, we provide a comprehensive review of LLM-based agent optimization approaches, categorizing them into parameter-driven and parameter-free methods. We first focus on parameter-driven optimization, covering fine-tuning-based optimization, reinforcement learning-based optimization, and hybrid strategies, analyzing key aspects such as trajectory data construction, reward function design, and optimization algorithms. Additionally, we briefly discuss parameter-free strategies that optimize agent behavior through prompt engineering and external knowledge retrieval. Finally, we summarize the evaluation for agents, review key applications of LLM-based

This work is supported by the National Natural Science Foundation of China (Grant Nos. 62477011, 62207013).

Authors' Contact Information: Shangheng Du, Shanghai Institute of Artificial Intelligence for Education, East China Normal University, Shanghai, China, School of Computer Science and Technology and East China Normal University, Shanghai, China; e-mail: dsh@stu.ecnu.edu.cn; Jiabao Zhao (corresponding author), School of Computer Science and Technology, Donghua University, Shanghai, China; e-mail: jbzhaod@dhru.edu.cn; Jinxin Shi, Shanghai Institute of Artificial Intelligence for Education, East China Normal University, Shanghai, China, School of Computer Science and Technology and East China Normal University, Shanghai, China; e-mail: jinxinshi@stu.ecnu.edu.cn; Zhentao Xie, School of Computer Science and Technology, East China Normal University, Shanghai, China; e-mail: ecnudavidtao@gmail.com; Xin Jiang, School of Computer Science and Technology, East China Normal University, Shanghai, China; e-mail: 51275901099@stu.ecnu.edu.cn; Yanhong Bai, School of Computer Science and Technology, East China Normal University, Shanghai, China; e-mail: Lucky_Baiyh@stu.ecnu.edu.cn; Liang He, School of Computer Science and Technology, East China Normal University, Shanghai, China; e-mail: lhe@cs.ecnu.edu.cn.



This work is licensed under a [Creative Commons Attribution 4.0 International License](https://creativecommons.org/licenses/by/4.0/).

© 2026 Copyright held by the owner/author(s).

ACM 0360-0300/2026/02-ART223

<https://doi.org/10.1145/3789261>

agents, and discuss the major challenges and promising future directions. A curated collection of the surveyed works is provided at <https://github.com/YoungDubbyDu/LLM-Agent-Optimization>.

CCS Concepts: • **Computing methodologies** → **Natural language processing**;

Additional Key Words and Phrases: Agent optimization, fine-tune, reinforcement learning, prompt engineering, large language models

ACM Reference Format:

Shangheng Du, Jiabao Zhao, Jinxin Shi, Zhentao Xie, Xin Jiang, Yanhong Bai, and Liang He. 2026. A Survey on the Optimization of Large Language Model-based Agents. *ACM Comput. Surv.* 58, 9, Article 223 (February 2026), 37 pages. <https://doi.org/10.1145/3789261>

1 Introduction

The development of autonomous agents has been a long-term pursuit in **Artificial Intelligence (AI)**. AI agents have evolved from early rule-based and expert system-based architectures to **reinforcement learning (RL)**-driven agents, which are now widely applied in many fields [34]. Traditional RL-based agents optimize policies through interaction with environments, using structured reward functions to achieve goals and improve performance over time. However, these approaches often require extensive training, rely on well-defined state and action spaces, and struggle with generalization across diverse tasks.

In recent years, **Large Language Models (LLMs)** such as GPT-5 [112] and Deepseek-r1 [49] have achieved remarkable success, demonstrating exceptional capabilities in language understanding, reasoning, planning, and complex decision-making. Building on these strengths, LLMs can serve as agents, providing a promising pathway to improve autonomous decision-making and achieve AGI [168]. Unlike conventional RL-based agents, which optimize explicit reward-driven policies, LLM-based agents operate through text-based instructions and prompt templates and **in-context learning (ICL)**, allowing greater flexibility and generalization. These agents leverage the comprehension and reasoning capabilities of LLMs to interact with environments through natural language, execute complex multi-step tasks, and dynamically adapt to evolving scenarios. Existing LLM-based agents utilize various methods such as task decomposition [60], self-reflection [128], memory augmentation [210], and multi-agent collaboration [80] to achieve high performance in a range of domains, including software development [61], embodied intelligence [212], web navigation [28].

However, despite their strengths, LLMs are not inherently designed for autonomous decision-making or long-term tasks. Their training objectives focus on next-token prediction rather than the reasoning, planning, or interactive learning required for agent-based behaviors. This distinction also separates LLM-based agents from tool-augmented reasoning models: While tool-augmented LLMs can call external functions, they remain fundamentally reactive, executing tools only when prompted. In contrast, LLM-based agents employ the model as a cognitive core within an agentic workflow, enabling the system to initiate actions, plan strategically, select tools autonomously, and regulate its own behavior through reflection. As a result, deploying LLMs as agents in complex environments presents several key challenges: (1) long-horizon planning and multi-step reasoning often degrade over extended interactions; (2) limited memory prevents effective use of past experience; and (3) reliance on static pre-training reduces adaptability to novel or dynamic environments. Such issues are especially pronounced in open-source LLMs, which still trail proprietary models like GPT-5 in agentic capability. At the same time, the high cost and opacity of closed-source systems further highlight the need for optimizing open LLMs for agent performance.

Existing techniques, such as **supervised fine-tuning (SFT)** [115] and **reinforcement learning with human feedback (RLHF)** [113], have made significant strides in improving LLM performance

in instruction following tasks, but they fail to fully address the challenges of decision-making, long-term planning, and adaptability for LLM-based agents. Optimizing LLM-based agents requires a deeper understanding of dynamic environments and agent behaviors, as well as the design of more suitable algorithms tailored to agent-specific demands. To address these challenges, recent studies have explored various optimization strategies that help LLM-based agents refine their decisions, self-evolve, and handle dynamic, complex tasks.

In this article, we provide a comprehensive survey on LLM-based agent optimization, systematically categorizing methods into parameter-driven and parameter-free optimization strategies. This survey focuses on methodologies for enhancing LLM-based agent capabilities through tuning, RL, and other optimization strategies. Specifically, **Parameter-driven Optimization** refers to adjusting LLM parameters to enhance agent performance. We categorize existing methods into three types: (1) conventional fine-tuning-based optimization, consisting of two key stages: agent trajectory construction and agent fine-tuning; (2) RL-based optimization, further divided into reward function-based methods leveraging traditional RL algorithms like Actor-Critic [145] and **Proximal Policy Optimization (PPO)** [131], and preference alignment-based methods utilizing **Direct Preference Optimization (DPO)** [127] to align agent policies with human preferences or task objectives; (3) hybrid fine-tuning optimization, an emerging direction that integrates SFT with RL to iteratively refine agent behaviors. In addition, we outline **Parameter-free Optimization** methods, which aim at improving agent behavior without modifying model parameters. These methods leverage prompt engineering, in-context learning and **retrieval-augmented generation (RAG)**, incorporating various types of information into prompts to guide agents' actions. We categorize them into experience-based, feedback-based, tool-based, Retrieval-based, and multi-agent collaborative optimization. To ensure reproducibility and facilitate practical use, we provide an extended list with representative agent optimization methods (including article links and open-source code repositories) at <https://github.com/YoungDubbyDu/LLM-Agent-Optimization>.

Comparison to related surveys. While LLM-based agents have attracted increasing attention, existing surveys primarily focus on general LLM optimization [105, 115, 146] or specific agent components such as planning [60], memory [210], and multi-agent coordination [80]. However, they do not treat LLM-based agent optimization as a distinct research topic. In contrast, this work is the first survey on LLM-based agent optimization techniques, facilitating a clearer understanding and comparison of existing methods and providing directions for future research.

Scope and rationales. (1) We survey only LLM-based agent optimization methods aimed at enhancing agent capabilities such as problem-solving and task execution, covering both parameter-driven and parameter-free approaches. We exclude works centered on general LLM efficiency, role-playing, or dialogue; (2) Our selection includes articles from top AI and NLP conferences and journals, as well as recent high-impact arXiv preprints to capture the latest developments. (3) We focus on studies since 2022 to reflect recent advancements in LLM-based agent optimization.

Our key contributions can be summarized as follows:

- We present a comprehensive survey focused on the optimization of LLM-based agents. To the best of our knowledge, this is the first systematic review dedicated to this line of research.
- We categorize optimization methods by their underlying techniques and analyze them in depth from both algorithmic and workflow perspectives, covering parameter-driven and parameter-free approaches.
- We summarize widely used open-source frameworks, evaluation, datasets, highlight representative applications across domains, and discuss current challenges and future directions.

Organization of this survey. The schematic layout of this manuscript is illustrated in Figure 1. Section 2 provides the background knowledge and related concepts. In Section 3, we systematically review parameter-driven optimization approaches. Section 4 classifies and summarizes

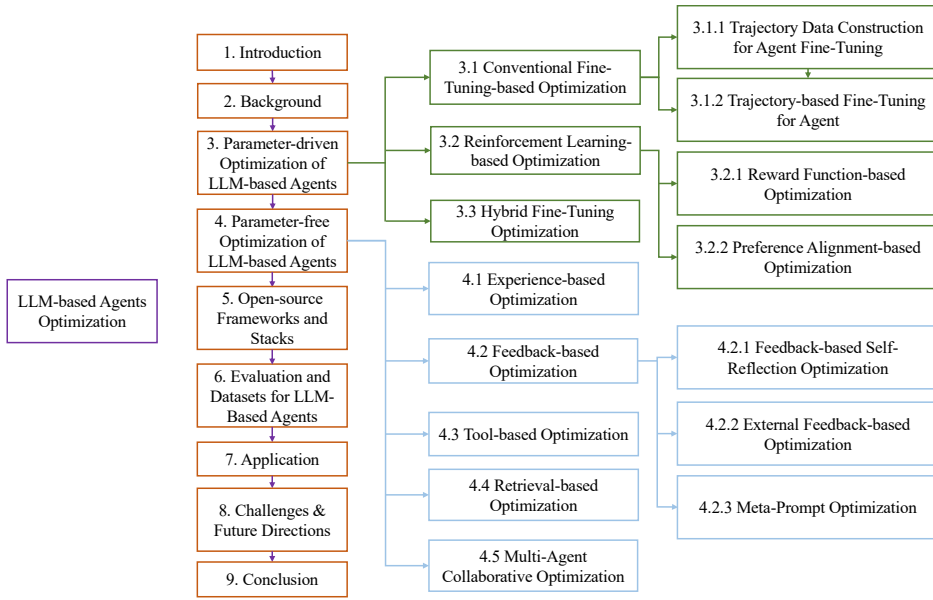


Fig. 1. An overview of the article organization.

parameter-free optimization methods. Section 5 introduces open-source agent frameworks and stacks. Section 6 presents datasets and evaluation benchmarks, while Section 7 reviews agent practical applications in various domains. Finally, Section 8 highlights challenges and future directions.

2 Background

2.1 Reinforcement Learning-based Agent Optimization

RL has long provided a foundation for agent optimization by enabling agents to learn from interactions with environments. Existing approaches primarily fall into value-based and policy-based methods [34, 96, 109]. Value-based algorithms such as Q-learning [24, 161] estimate action values to guide decision-making, while policy-based algorithms, including Policy Gradient and PPO [45, 117, 131], directly optimize policies through reward gradients, often improving stability via constrained updates. Actor-Critic methods [145] integrate value estimation with policy learning to enhance convergence. Beyond single-agent settings, **Multi-Agent RL (MARL)** extends these principles to cooperative and competitive multi-agent interaction [10, 203].

Recent developments increasingly focus on aligning agent behavior with human preferences. RLHF [113] incorporates human feedback into policy optimization, and DPO [127] further streamlines preference-based learning by removing explicit reward modeling. Overall, RL-based optimization has progressed from classical value and policy learning to preference-driven and multi-agent formulations, offering essential foundations for decision-making in LLM-based agents.

2.2 LLM Fine-Tuning

LLM fine-tuning is a key technique for adapting pre-trained models to specific tasks through optimizing parameters. The most widely used approach is SFT, where LLMs are trained on labeled data to improve task-specific performance. Instruction tuning, a common variant of SFT, enhances LLMs' ability to follow human instructions by training on instruction-response pairs [90, 205].

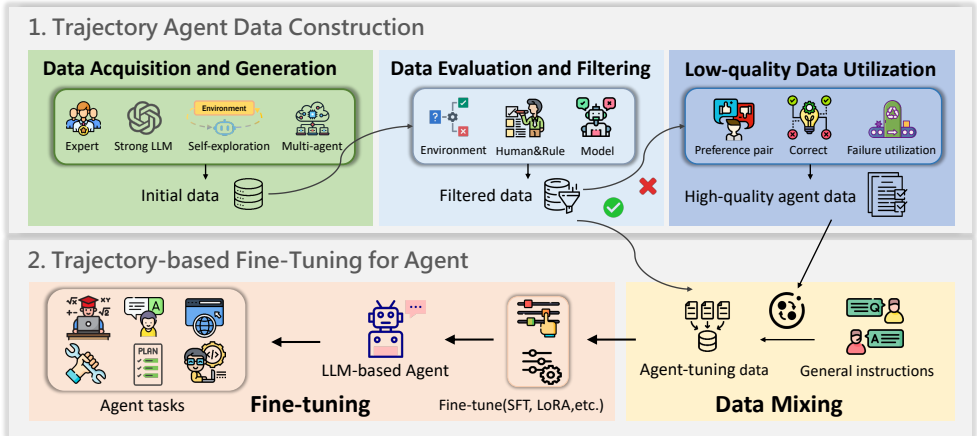


Fig. 2. Workflow of fine-tuning-based optimization for LLM-based agents.

Another major development is **parameter-efficient fine-tuning (PEFT)**, which updates only a small subset of parameters to significantly reduce computational cost while preserving performance. Representative methods include P-Tuning [94], LoRA [55], and QLoRA [30]. Additionally, RLHF is used to fine-tune LLMs by integrating human feedback, improving their decisions and aligning outputs with user preferences [113]. These techniques enable LLMs to adapt more efficiently to a wide range of tasks, enhancing their effectiveness in real-world scenarios.

3 Parameter-driven Optimization of LLM-based Agents

Comparison with LLM parameter optimization. Parameter-driven LLM optimization focuses on “how to create a better model”, aiming at enhancing general language understanding, instruction following, and broad task performance. In contrast, LLM-based agent parameter optimization addresses “how to better use the model to solve complex agent tasks”, emphasizing task-specific capabilities such as decision-making and multi-step planning in dynamic environments.

In this section, we discuss how parameter-driven optimization methods improve the performance of LLM-based agents. Specifically, we categorize these methods into three main technical approaches according to different strategies for parameter tuning: conventional fine-tuning-based optimization, RL-based optimization, and hybrid optimization.

3.1 Conventional Fine-Tuning-based Optimization

Conventional fine-tuning-based agent optimization adapts LLM parameters through instruction tuning and PEFT methods, using trajectory datasets typically formatted as SFT to align models with task-specific behaviors. This process generally involves two components: (1) constructing high-quality trajectories tailored to agent tasks, and (2) fine-tuning the LLM-based agent (workflow shown in Figure 2). Methods involving RL-based optimization (e.g., PPO, DPO) are discussed separately in Section 3.2. Here, we focus on traditional fine-tuning techniques employed in existing agent-optimization systems, outlining how each stage contributes to improving agent capabilities. Table 1 summarizes representative approaches, highlighting data preparation and fine-tuning strategies.

3.1.1 Trajectory Data Construction for Agent Fine-Tuning. Previous studies [39, 77, 115] have shown that the quality of training data significantly impacts model performance, making trajectory construction a critical step in the fine-tuning pipeline. This process typically involves trajectory

Table 1. Comparison of Conventional Fine-Tuning-based Optimization for LLM-based Agents: Data Construction and Fine-Tuning

Method	Trajectory Agent Data Construction				Fine-Tuning	
	Generation	Filtering	MA	LQ	Fine-tune Approach	Base Model
AgentTuning [197]	Strong LLM	Human or Rule	✓	✓	Instruction Tuning	Llama2-7B/13B/70B
SMART [195]	Multi-agent	Environment	/	✓	LoRA	Llama-2-7B
Agent-FLAN [21]	Expert	Model	✓	✓	Instruction Tuning	Llama-2-7B
Self-Talk [151]	Multi-agent	Human or Rule	/	✓	LoRA	MosaicAI-7B-Chat
ENVISIONS [178]	Self-exploration	Environment	/	✓	Customized	Llama2-7B/13B-Chat
AgentGym [169]	Strong LLMandExpert	Environment	/	✓	BC	Llama-2-7B-Chat
FireAct [13]	Strong LLM	Environment	/	/	LoRA	GPT3.5, Llama-2-7B/13B, CodeLlama-7B/13B/34B-Instruct
NAT [158]	Strong LLM	Environment	/	✓	SFT	Llama-2-7B/13B-Chat
Agent Lumos [191]	Strong LLM	Human or Rule	/	/	LoRA	Llama-2-7B/13B
STE [152]	Self-exploration	Model	/	✓	SFT	Llama-2-7B/13B-Chat, Mistral-7B-Instruct
OPTIMA [18]	Multi-agent	Human or Rule	✓	/	SFT	Llama-3-8B
Zhou et al. [217]	Strong LLM	Human or Rule	✓	/	LoRA	OpenChat v3.2, Llama-2-7B, AgentLM-7B
AgentOhana [202]	Expert	Model	/	/	QLoRA	xLAM-v0.1
COEVL [78]	Expert	Model	✓	/	SFT	Llama-2-7B, Mistral-7B
AgentBank [140]	Strong LLM	Environment	/	✓	Instruction Tuning	Llama-2-Chat
AdaSwitch [144]	Self-exploration	Model	✓	✓	SFT	DeepSeek-Coder-1.3B, StarCoder2-3B
IPR [177]	Expert and Self-exploration	Environment	/	✓	Instruction Tuning	Llama-2-7B
Re-ReST [32]	Self-exploration	Environment	/	✓	LoRA	Llama-2-7B/13B, Llama-3-8B, CodeLlama-13B, VPGen
ATM [220]	Multi-agent	/	✓	/	MITO (Customized)	Llama-2-7B
Aksitov et al. [2]	Self-exploration	Model	/	/	SFT	PaLM-2-base-series
SWIFTSAGE [86]	Self-exploration	Environment	✓	/	SFT	T5-Large
AGILE [38]	Expert	/	/	/	BC	Vicuna-13B, Meerkat-7B
NLRL [39]	Self-exploration	/	/	/	SFT	Llama-3.1-8B-Instruct
ETO [141]	Expert	/	/	✓	BC	Llama-2-7B-Chat
Retrospec [171]	Expert	/	/	✓	BC	Flan-T5-Large, Llama-3-8B-Instruct
ToRA [46]	Strong LLM	Human or Rule	/	✓	BC	Llama-2-series, CodeLlama-series
Sayself [180]	Strong LLM	Human or Rule	/	/	SFT	Mistral-7B, Llama-3-8B
Star-Agents [215]	Strong LLM	Model	✓	/	SFT	Pythia-1B, Llama-2-7B
ATLaS [20]	Expert	Model	/	/	Customized	Mistral-7B, Llama-3-8B
WKM [123]	Expert	Model	✓	/	LoRA	Mistral-7B-Instruct, Llama-3-8B-Instruct, Gemma-1.1-7B
Subramaniam et al. [143]	Multi-agent	Model	✓	/	SFT	Phi-3-Mini-128K-Instruct, Mistral-7B-Instruct, Llama-3-8B-Instruct
E ² CL [154]	Expert& Self-exploration	Human or Rule	/	✓	SFT	Flan-t5-small/base/large
STeCa [155]	Expert	/	/	✓	SFT	Llama-2-7B-Chat, Llama-3-8B-Instruct, Mistral-7B

Note: MA - Multi-Agent Framework; LQ - Low-Quality Data Utilization.

data generation, evaluation and filtering, and utilization of low-quality samples, to construct refined data that meet the requirements for effective fine-tuning.

Data Acquisition and Generation. High-quality trajectory construction begins with the acquisition and generation of initial data, which require not only diversity but also strong task alignment to ensure effective learning. These methods can generally be classified into four broad categories: expert-annotated data, strong LLM-generated trajectories, self-exploration environment-interaction trajectories, and multi-agent collaboration-based construction.

(1) Expert-annotated data. Expert-annotated trajectories provide high-quality, human-curated demonstrations and are widely regarded as the gold standard for agent fine-tuning. Such data ensure strong task alignment because experts can design thoughts, observations, and actions [188] that model the intended decision-making process. Many systems [13, 38, 141, 155, 158, 177] adopt ReAct-style trajectories as foundational training data, with methods like IPR [177], ETO [141], and AGILE [38] using CoT-enhanced demonstrations [162] to reinforce task-specific reasoning skills. To improve compatibility with pre-trained LLM distributions, Agent-FLAN [21] restructures ReAct trajectories into multi-turn dialogue formats, while AgentOhana [202] further standardizes heterogeneous expert trajectories to improve consistency.

Table 2. Comparison of Data Acquisition and Generation Strategies

Strategy	Advantages	Limitations
Expert-annotated	Ensures high accuracy with flexible, task-specific manual design.	Labor-intensive, limited, and difficult to obtain in professional domains.
Strong LLM-generated	Provides stable and high-performance trajectories without complex design.	Incurs high usage cost and may introduce potential biases and factual inconsistencies.
Self-exploration	Offers low cost, easy implementation, and no need for extra assistance.	Easily produces low-quality trajectories with errors, requiring effective data filtering.
Multi-agent collaboration	Improves accuracy and scalability through task specialization.	Relies on complex coordination mechanisms, which increase design complexity and cost.

(2) **Strong LLM-generated trajectories.** Strong LLM-generated trajectories rely on powerful models such as ChatGPT or GPT-4 to autonomously produce task-specific data, typically using reasoning frameworks like ReAct and CoT to simulate multi-step reasoning, decision-making, and interaction with environments. Many approaches, including AgentTuning [197] and FireAct [13], employ ReAct-, CoT-, or Reflexion-style prompting [136] to improve trajectory diversity and reasoning quality. Other works enrich trajectories with tool use or structured annotations, such as NAT [158], which incorporates calculators and APIs, and Agent Lumos [191], which uses GPT-4/GPT-4V to produce annotated planning and grounding data. Multi-role simulation has also been explored, with systems like Zhou et al. [217] using GPT-4 to coordinate problem generators, planners, and environment agents to iteratively generate more complex trajectories.

(3) **Self-exploration environment-interaction trajectories.** Given the high cost of proprietary models like GPT-4, self-exploration with open-source LLMs has become a common strategy for generating trajectory data. These approaches allow agents to interact with environments and improve trajectories through autonomous exploration, feedback, and self-training. Early methods [2, 86] directly collect trajectories from environment interactions, such as SWIFTSAGE [86], which samples actions based on state-history pairs to build concise datasets. More recent work incorporates feedback and reflection, with systems such as ENVISIONS [178] and STE [152] enabling self-correction, tool/API simulation, and experience storage to refine generated trajectories. Other approaches use auxiliary models to guide refinement: AdaSwitch [144] employs a stronger cloud LLM to correct errors made by a local model, while Re-ReST [32] iteratively improves outputs through a reflector model. Natural language-based RL frameworks like NLRL [39] further unify policies, values, and feedback in text form, enabling trajectory generation from an RL perspective.

(4) **Multi-agent collaboration-based construction.** Multi-agent collaborative frameworks enhance trajectory diversity and robustness by incorporating role assignment and coordinated interactions that go beyond single-agent generation. Although often built on strong LLMs or self-exploration, their collaborative mechanisms merit separate attention. Most approaches [78, 151, 195, 220] assign complementary roles and let agents iteratively generate or refine trajectories through sequential or alternating interactions. For example, SMART [195] organizes agents as intent reconstructor, knowledge retriever, and response generator to co-produce multi-turn trajectories, while COEVOL [78] uses advisor-editor-judge roles in a debate-style refinement loop. To improve robustness, adversarial collaboration frameworks such as ATM [220] pair an attacker perturbing retrieved evidence with a generator trained to produce disturbance-resistant outputs.

In conclusion, data acquisition and generation strategies form the foundation of trajectory construction for LLM-based agents, with each approach offering distinct benefits and limitations, as summarized in Table 2. Their suitability varies with task demands, reflecting tradeoffs among quality, scalability, and complexity. Future work may benefit from hybrid strategies that combine complementary methods to better leverage their respective strengths.

Table 3. Comparison of Data Evaluation and Filtering Strategies

Strategy	Advantages	Limitations
Environment-based	Easy to implement with clear and well-defined reward signals.	Primarily focuses on final binary outcomes, neglecting step feedback and potentially accumulating errors.
Human or Rule-based	High reliability, strong interpretability, and flexible customization.	Relies on predefined metrics or human oversight, and requires complex rule design.
Model-based	Fully automated process, reducing human effort and manual design.	Depends on model reliability and is affected by inherent biases within the model.

Data Evaluation and Filtering. After trajectory data are generated, evaluating and filtering it becomes crucial for ensuring fine-tuning quality. While expert-annotated datasets are often directly usable, other trajectory types typically require additional refinement through criteria-based assessment, external feedback, or model scoring. We group existing filtering approaches into three categories: environment-based, human or rule-based, and model-based evaluation.

(1) Environment-based. Environment-based methods rely on feedback from the environment, such as environmental rewards or task completion signals, to assess agent actions and filter trajectory data. Most works [13, 169, 178, 197] in this category adopt a form of binary feedback, where the reward is assigned based on the success or failure of the agent’s actions relative to the desired task. AgentTuning [197], ENVISIONS [178], and FireAct [13] use environment-based rewards to evaluate trajectory success, relying on environmental signals or correctness assessments. Similarly, NAT [158] also employs a binary environment-based reward system, differentiating between successful and unsuccessful outcomes to encourage agents to learn from failure.

(2) Human or Rule-based. Human or rule-based evaluation assesses trajectory quality through predefined criteria, manual inspection, or task-specific rules, focusing on factors such as correctness, diversity, and consistency. Many methods [2, 191, 217] adopt such approaches to ensure alignment with task requirements. Typical strategies include manual filtering to verify consistency between environment feedback and agent actions [217], or perplexity-based selection to retain samples most aligned with task objectives [2]. Other systems [18, 151, 178] employ multi-dimensional criteria—e.g., dialogue diversity, sub-goal completion, or readability to select higher-quality trajectories, with some methods ranking trajectories using reward or probability-based scoring [178].

(3) Model-based. Model-based evaluation leverages LLMs (e.g., GPT-3.5/4) to automate trajectory assessment using their reasoning and scoring capabilities. Such methods typically rely on LLMs to rank or validate trajectories for relevance, correctness, and completeness. For example, [2] and AdaSwitch [144] employ LLM-based scoring to compare multiple samples and verify stepwise correctness, while STE [152] uses GPT-4 to assess examples involving tool or API usage. Beyond input–output checks, several approaches evaluate full trajectories: AgentOhana [202] applies an AgentRater to score process integrity and outcome validity, retaining only high-quality trajectories, and COEVOL [78] uses GPT-3.5 for comparative evaluation across criteria such as helpfulness, relevance, and accuracy.

In summary, different evaluation and filtering strategies vary in their advantages and limitations, as summarized in Table 3. These methods offer distinct tradeoffs in scalability, accuracy, and implementation complexity, requiring careful selection and design based on agent task-specific demands. Integrating multiple approaches into a hybrid strategy can further enhance data quality and robustness, ensuring a more reliable evaluation framework tailored to different task needs.

Low-quality Data Utilization. Because filtering often yields limited high-quality data, leveraging low-quality or failed trajectories has become an effective strategy for dataset augmentation. Early approaches relied primarily on expert-crafted or successful trajectories, overlooking the

informative value of failures. Subsequent methods address this in two ways. Several works [169, 177, 212] construct comparative datasets by pairing successful and failed trajectories, enabling agents to learn from both correct and incorrect behaviors. Others focus on refining low-quality samples: AgentBank [140] regenerates failed interactions, while Re-ReST [32] and AdaSwitch [144] iteratively correct errors using environment or model feedback. In contrast, some approaches use error samples directly to teach failure recognition. Agent-FLAN [21] integrates frequent negative examples to reduce hallucinations, and NAT [158] generates deliberately incorrect responses to strengthen the model's ability to distinguish valid from invalid outputs.

3.1.2 Trajectory-based Fine-Tuning for Agent. Fine-tuning is a critical step in the optimization process for LLM-based agents, enabling open LLMs to adapt to specific agent tasks or data distributions. Most techniques are based on standard LLM fine-tuning methods, but are specifically applied to agent-related data (constructed in Section 3.1.1) to enhance the decision-making and task completion.

Mixing of General Instructions and Agent Trajectories. Some studies [21, 217] have shown that fine-tuning solely on agent-specific trajectories may weaken general language and reasoning abilities of LLM-based agents, so it is common to mix general instruction datasets and task-specific trajectories during fine-tuning to preserve foundational skills while optimizing for agent-specific tasks. For example, AgentTuning [197] combines AgentInstruct datasets with open-source instructions to fine-tune LLaMA-2 models, while AgentBank [140] integrates agent trajectories, general instruction data, and code datasets to optimize LLaMA-2-Chat models for diverse tasks.

The Usage of Fine-Tuning Techniques. Based on studies on LLM-based agents, we categorized fine-tuning methods into three types: standard SFT, PEFT (e.g., LoRA), and customized strategies tailored for specialized tasks, which will be discussed in detail.

(1)Standard SFT. The goal of SFT in LLM-based agents is to align pre-trained models with specific task requirements by minimizing the discrepancy between predicted and target outputs. Here, standard SFT refers to full-parameter fine-tuning on high-quality training data, including instruction tuning. Since **Behavior Cloning (BC)** in imitation learning follows the same paradigm, we also include it in this category. Most works adopt standard SFT to fine-tune LLM-based agents using high-quality datasets. AgentTuning [197], Agent-FLAN [21], and AgentBank [140] fine-tune Llama-based models with instruction data to align them with task requirements. Similarly, NAT [158], STE [152], and COEVOL [78] employ SFT on trajectory datasets capturing both successful and failed task cases. In addition, AGILE [38], Retrospec [171], AgentGym [169], ETO [141], and ToRA [46] adopt BC-style SFT to equip models with foundational agent capabilities. Overall, the simplicity and effectiveness of standard SFT make it a core method for aligning LLMs with task-specific behaviors.

(2)Parameter-efficient fine-tuning. PEFT methods, such as LoRA and QLoRA, update only a small number of parameters while keeping most of the LLM frozen, significantly reducing computational and memory costs and enabling efficient fine-tuning of large models. Many works, including SMART [195], FireAct [13], Re-ReST [32], and Agent Lumos [191], apply LoRA to fine-tune models from the Llama series, OpenChat, and CodeLlama across various configurations. AgentOhana [202] further uses QLoRA to optimize xLAM-v0.1 models, demonstrating effective fine-tuning under resource constraints. Overall, PEFT provides a practical solution for optimizing LLM-based agents, balancing between performance and efficiency in limited-resource settings.

(3)Customized fine-tuning. Customized fine-tuning methods are tailored to specific tasks or ideas, incorporating designed strategies or objective functions, such as introducing additional regularization constraints, to better align agents with task requirements. ATM [220] designs **Multi-agent Iterative Tuning Optimization (MITO)** loss, which combines standard SFT with KL regularization to balance task-specific optimization and generalization. Leveraging contrastive

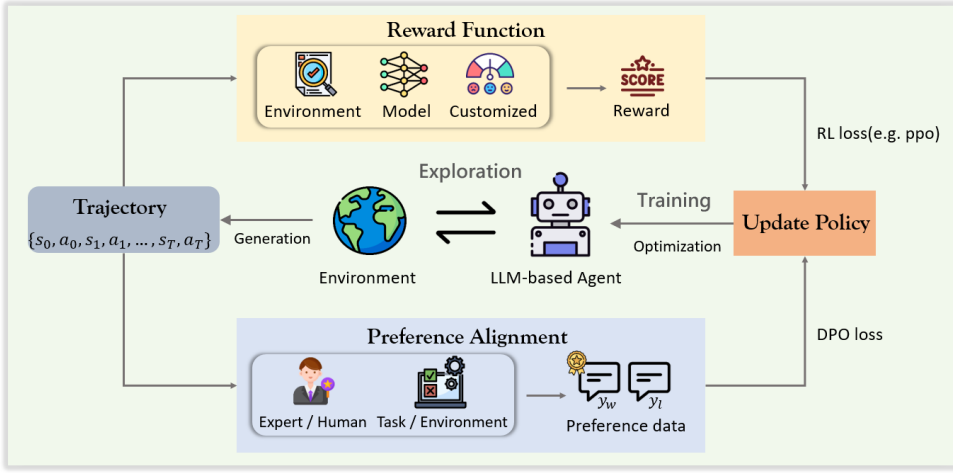


Fig. 3. Workflow of reinforcement learning-based optimization for LLM-based agents.

learning and RL-free optimization, ENVISIONS [178] achieves efficient iterative refinement of Llama-2 through cyclic fine-tuning on dynamically updated trajectory pairs. These methods highlight the flexibility of adapting LLMs to agent-specific tasks by customized objectives and iterative optimization.

3.1.3 Summary. Conventional fine-tuning-based optimization has effectively enhanced LLM-based agents by high-quality trajectory data, different training strategies, and efficient parameter updates, ensuring strong task alignment, controlled optimization, and adaptability to specific objectives. However, such methods also face inherent limitations due to their dependence on curated data, vulnerability to overfitting, and limited adaptability to dynamic environments. Additionally, without interactive feedback mechanisms, they may accumulate errors and struggle with generalization, because fine-tuning typically aligns models with static objectives and restricts real-time behavioral refinement. With the growing complexity and dynamics of agent tasks, exploring more adaptive optimization strategies becomes essential to further enhance their decision-making capabilities.

3.2 Reinforcement Learning-based Optimization

RL-based optimization has emerged as a promising approach, enabling LLM-based agents to learn directly from interactions with environment or human feedback and dynamically refine their actions through rewards. Unlike static fine-tuning, RL encourages exploratory learning, allowing agents to discover novel policies and adapt to unseen tasks. By aligning model outputs with explicit rewards or preferences, RL enhances both task performance and robustness in complex, evolving environments. We categorize RL-based optimization methods into two primary approaches: **Reward-function-based optimization** and **Preference-alignment-based optimization**. The overall workflow is illustrated in Figure 3.

3.2.1 Reward Function-based Optimization. Reward function-based optimization methods leverage explicit reward signals to refine the behavior of LLM-based agents, facilitating adaptation to complex and dynamic tasks. Drawing from traditional RL paradigms, these methods employ algorithms like PPO to iteratively optimize agents' policies by adjusting LLMs' parameters. By treating LLM as an agent, these approaches utilize diverse reward sources, including environmental feedback, model-generated signals, and customized reward functions, as summarized in Table 4.

Table 4. Summary of Reward Function-based Optimization Methods for LLM-based Agents

Method	Reward Source	RL Algorithm	MA	Base Model
CMAT [84]	Environment-based	Actor-Critic	✓	Tinyagent-1.8B/7B
StepAgent [29]	Model-based	DPO+PPO	/	Mistral-7B, Llama3-8B
WebRL [119]	Model-based	Actor-Critic	✓	GLM-4-9B, Llama-3.1-8b/70B
CORY [101]	Customized, Model-based	PPO	✓	GPT 2-Large, Llama-2-7B-Chat
SaySelf [180]	Customized	PPO	✓	Mistral-7B, Llama-3-8B
AgentGym [169]	Environment-based	AgentEvol	/	Llama-2-7B-Chat
GELI [70]	Model-based	PPO	/	Llama-2-7B
AGILE [38]	Customized	PPO	/	Vicuna-13B, Meerkat-7B
RISE [126]	Environment-based	RWR	/	Llama-2-7B-chat, Mistral-7B-Instruct
KALM [114]	Environment-based	CQL	/	Llama-2-7B-Chat-hf
RL4VLM [200]	Environment-based	PPO	/	Llava-v1.6-mistral-7B
POAD [163]	Environment-based	POAD	/	Llama-3.1-8B-Instruct
STeCa [155]	Customized, Model-based	PG	/	Llama-2-7B-Chat, Llama-3-8B-Instruct, Mistral-7B
DeepResearcher [214]	Customized	GRPO	/	Qwen2.5-7B-Instruct

Note: MA- Multi-Agent Framework.

Environment-based rewards. One of the most common approaches leverages environmental feedback as the primary reward source. CMAT [84] uses an actor-critic framework in multi-agent collaboration, where agents interact with environments and receive rewards based on task outcomes. Retrospec [171] applies offline implicit Q-Learning to stabilize policy updates by minimizing **temporal-difference (TD)** error on fixed trajectories. AgentGym [169] optimizes itself using the AgentEvol algorithm guided by environment rewards. Environment-based rewards are straightforward to obtain, but are often discrete and outcome-focused, which can limit their ability to provide detailed guidance during intermediate steps of complex tasks.

Model-based rewards. Model-based rewards derive signals from models trained to evaluate agent behaviors, providing implicit or explicit feedback to guide policy optimization. These methods are particularly useful when direct environmental rewards are sparse or unavailable. StepAgent [29] employs an inverse RL framework in which a discriminator predicts how well trajectories align with expert behaviors and provides fine-grained step-wise rewards for PPO optimization. WebRL [119] trains a self-supervised Optimal Reward Model to evaluate trajectory quality; using an actor-critic framework, it updates policy weights with ORM-generated rewards and applies KL constraints for policy stability. In summary, model-based rewards offer fine-grained, step-level feedback that enhances generalization beyond final outcome rewards. However, their effectiveness depends on the robustness of the reward models, which require careful design and optimization.

Customized reward functions. Customized reward functions adapt reinforcement learning frameworks to the specific objectives of agent tasks, extending optimization signals beyond binary task success to factors such as stability, efficiency, and reliability. GELI [70] integrates global rewards with local multimodal feedback to guide long-horizon conversational behavior using PPO. AGILE [38] designs a reward balancing task completion and expert-query costs, encouraging strategic assistance-seeking. CORY [101] employs shared rewards and KL constraints in a multi-agent setting to improve policy quality, while SaySelf [180] calibrates model confidence by rewarding accurate predictions and penalizing overconfident errors. Recent advances extend these ideas to more complex reasoning. DeepResearcher [214] introduces an end-to-end RL framework in real web environments, enabling LLM agents to iteratively reason, search, and synthesize information for open-domain questions. Collectively, these works illustrate the flexibility of RL-based optimization through task-specific reward designs tailored to diverse agent capabilities.

Table 5. Summary of Preference Alignment-based Optimization Methods for LLM-based Agents

Method	Preference Criteria	Algorithm	MA	Base Model
StepAgent [29]	Expert	DPO+PPO	/	Mistral-7B, Llama-3-8B
Agent Q [118]	Environment	DPO	/	Llama-3-70B, xLAM-v0.1-r
ETO [141]	Environment	DPO	/	Llama-2-7B-Chat
OPTIMA [18]	Task, Human	DPO	✓	Llama-3-8B
IPR [177]	Expert, Environment	DPO	/	Llama-2-7B
Re-ReST [32]	Environment	DPO	✓	Llama-2-7/13B, Llama-3-8B, Codellama-13B, VPGen
DMPO [135]	Expert	DMPO	/	Llama-2-7B-Chat, Mistral-7B-Instruct-v0.2
EPO [212]	Task, Environment	DPO	✓	Llama-2-7B
ATM [220]	Human	DPO	✓	Mistral-7B-Chat
AMOR [48]	Expert, Human	KTO	✓	Llama-2-7B/13B-Chat
DITS [134]	Environment, Human	DPO	✓	Llama-3-8B-Instruct

Note: MA- Multi-Agent Framework.

3.2.2 Preference Alignment-based Optimization. Preference alignment-based optimization offers an alternative to traditional RL by aligning model behavior with human or other preferences without relying on explicit reward signals. Algorithms such as DPO can be viewed as RL variants tailored for offline datasets, where preferences replace reward signals to guide policy refinement. This offline formulation enables effective tuning of large-scale LLM-based agents without online rollout. The optimization process typically consists of two stages: **Preference Data Construction** and **Policy Optimization**, as follows:

- **Preference Data Construction.** The first step is constructing preference data in the form of pairwise comparisons, represented as tuples (x, y_w, y_l) , where x is the input instruction, y_w denotes the preferred (win) response, and y_l is the less preferred (lose) response (Some studies alternatively use a^+ and a^-). This structured format explicitly ranks outputs, forming a clear basis for preference-based optimization. Methods such as **Monte Carlo Tree Search (MCTS)** are often employed to generate diverse trajectories, which are then evaluated using task success rates, environment feedback, or other metrics to determine y_w and y_l .
- **Policy Optimization via DPO.** Following the construction of preference data, the agent's policy is optimized to align with these preferences. DPO is a widely adopted approach that directly defines a preference loss based on policy, optimizing LLM-based agents to prioritize desired outputs and adhere to task-specific requirements. The optimization objective of DPO is formalized as follows:

$$\mathcal{L}_{DPO}(\pi_\theta; \pi_{ref}) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w | x)}{\pi_{ref}(y_w | x)} - \beta \log \frac{\pi_\theta(y_l | x)}{\pi_{ref}(y_l | x)} \right) \right], \quad (1)$$

where $\mathcal{D}$ is the preference dataset, $\pi_\theta(y | x)$ denotes the probability assigned by the current policy to output y given input x , and $\pi_{ref}(y | x)$ is output probability under a frozen reference model. The regularization term β ensures the updates remain consistent with the original policy and avoids significant deviations from the reference model, while σ is the sigmoid function ensuring stable and efficient optimization.

We summarize representative works under this paradigm, categorized by preference criteria, including those based on human and task feedback, as shown in Table 5.

Optimization Based on Expert or Human Preferences. Preference data in this category rely on expert trajectories or human-defined rules, constructing positive samples y_w from ideal or successful behaviors and negative samples y_l from suboptimal trajectories. DMPO [135] refines

RL objectives by introducing state–action occupancy constraints from expert trajectories, guiding agents to mimic expert distributions; with length normalization, DMPO directly maximizes the likelihood of preferred trajectories to reduce multi-turn errors. IPR [177] builds step-level preference data by comparing agent actions with expert demonstrations and optimizes both outcome-DPO and step-DPO losses for fine-grained alignment. AMOR [48] adopts a two-stage approach, combining SFT pre-training with **Kahneman–Tversky Optimization (KTO)**. In the adaptive stage, human-annotated binary feedback is used to optimize module-specific parameters. Although expert and human-based preferences ensure high-quality data and accurate optimization, they are limited in scalability and coverage, as expert trajectories may not sufficiently represent diverse or unexplored scenarios in complex environments.

Optimization Based on Task or Environment Preferences. This category constructs preference datasets using task-specific metrics or environment feedback, leveraging rewards, success rates, or other performance indicators to form preference pairs for DPO optimization. Incorporating environment-specific signals improves adaptability to dynamic and task-oriented contexts. Methods such as ETO [141] and Re-ReST [32] directly use task success or environment feedback to build preference data. EPO [212] ranks outputs via a reward model and combines DPO with token-level alignment loss for stable training and precise task alignment. Structured methods include Agent Q [118], which uses MCTS to explore action trajectories and generate preferences from process feedback and success rates. OPTIMA [18] ranks MCTS node pairs by common ancestry and reward differences, selecting top-ranked pairs for DPO optimization. DITS [134] adopts a similar strategy and introduces a data influence metric to jointly score node pairs.

3.2.3 Summary. RL-based optimization methods have shown significant potential in enhancing LLM-based agents. By incorporating reward signals or preference alignment, these methods enable agents to adapt to complex and dynamic tasks. The choice of method often depends on task complexity and data availability: reward-function-based techniques excel in challenging scenarios but require substantial data and computation, whereas preference-alignment approaches simplify training and better match expected outputs but depend heavily on the quality and coverage of preference data, limiting adaptability in highly dynamic settings. Designing effective reward functions or preference pairs also requires careful consideration to capture task-relevant factors and avoid overly narrow criteria. These challenges underscore the need for progress in algorithmic efficiency and reward signal design. Moreover, current RL approaches remain limited in supporting long-horizon, multi-turn interactions, highlighting the need for more general and scalable training pipelines. Recent frameworks such as AgentGym-RL [170] reflect this direction by providing unified environments and multi-turn RL interfaces that enable more reliable long-horizon decision-making for LLM-based agents.

3.3 Hybrid Fine-Tuning Optimization

While traditional SFT provides a stable method for initializing LLM-based agents, it often struggles with dynamic tasks requiring complex decision-making. On the other hand, RL excels in exploring complex scenarios but typically demands extensive data and computational resources. To overcome these limitations, hybrid fine-tuning optimization combines the strengths of both SFT and RL, creating a more flexible and effective framework. This section introduces representative works that exemplify these hybrid approaches, as summarized in Table 6. We categorize them into two groups: sequential hybrid fine-tuning methods and non-sequential hybrid fine-tuning methods.

Sequential hybrid fine-tuning. Most works in hybrid fine-tuning follow a sequential approach, starting with a warm-up stage where behavior cloning-based SFT is applied to equip LLMs with foundational capabilities using high-quality datasets, such as expert trajectories. In the subsequent RL stage, algorithms like PPO or DPO refine the agent’s policy for task-specific objectives or dynamic

Table 6. Summary of Hybrid Fine-tuning Optimization Methods for LLM-based Agents

Method	Fine-tune Algorithm	RL algorithm	Hybrid Strategy	MA	Base Model
ETO [141]	BC	DPO	Sequential	/	Llama-2-7B-Chat
AGILE [38]	BC	PPO	Sequential	/	Vicuna-13B, Meerkat-7B
AMOR [48]	SFT	KTO	Sequential	✓	Llama-2-7B/13B-Chat
STeCa [155]	SFT	PG	Sequential	/	Llama-2-7B-Chat, Llama-3-8B-Instruct, Mistral-7B
SaySelf [180]	SFT	PPO	Sequential	✓	Mistral-7B, Llama-3-8B
RL4VLM [200]	SFT	PPO	Sequential	/	Llava-v1.6-mistral-7B
OPTIMA [18]	SFT	DPO	Non-sequential	✓	Llama-3-8B
IPR [177]	SFT	DPO	Non-sequential	/	Llama-2-7B

Note: MA- Multi-Agent Framework.

environments. This paradigm, similar to **Reinforcement Fine-Tuning (RFT)** introduced by OpenAI, has gained widespread popularity for combining SFT and RL to enhance LLMs' capabilities. For example, ReFT [150], ETO [141], Re-ReST [32], AGILE [38], and AMOR [48] all utilize SFT to train models for the warm-up phase. Following initialization, these methods apply various RL strategies. ETO and Re-ReST use DPO for preference alignment, AGILE employs PPO for decision-making, and AMOR applies KTO with binary feedback to align outputs with human preferences.

Non-sequential hybrid fine-tuning. Other studies adopt non-sequential strategies to refine agent's policy, including alternating and dynamically adjusted optimization between SFT and RL phases. OPTIMA [18] alternates SFT and DPO in the iterative training process, named as iSFT-DPO, enabling LLM-based agent to learn from optimal trajectories through SFT while refining its understanding using DPO based on comparative preferences. Similarly, IPR [177] incorporates step-level rewards, starting with ReAct-style expert trajectories for SFT and iteratively refining the policy through Outcome-DPO and Step-DPO losses.

Summary. Hybrid fine-tuning strategies combine the strengths of SFT and RL, allowing LLM-based agents to balance structured guidance with adaptive optimization. Although these methods offer greater flexibility for complex and dynamic tasks, they also face challenges such as increased computational costs and reliance on high-quality preference or reward data. Future research should explore more efficient hybrid designs to enhance the performance and versatility of LLM-based agents, making this an essential direction to advance agent optimization.

4 Parameter-free Optimization of LLM-based Agents

Besides parameter-driven optimization methods critical for evolving LLM-based agents, parameter-free optimization offers a promising alternative by optimizing the agent's behavior without modifying LLM parameters. It improves performance through prompt engineering techniques or incorporating various information, such as feedback, tools and external knowledge, making it especially efficient in resource-constrained environments. Unlike other surveys [60, 168, 210] that categorize methods according to agent structure or architecture, we categorize the methods based on the strategies and techniques employed to optimize agent behaviors, as summarized in Table 7.

4.1 Experience-based Optimization

Experience-based optimization leverages historical data, trajectories, or accumulated knowledge to improve LLM-based agents by deriving insights from past successes and failures, often organized through memory modules. This approach refines strategies, enhances long-term decision-making, and enables agents to generalize across tasks and domains.

Optimus-1 [82] utilizes a multimodal memory module to convert exploration trajectories into hierarchical knowledge graphs, which assists in task planning and prompt generation for the

Table 7. Comparison of Parameter-Free Optimization Methods

Optimization Strategy		Description	Example Works
Experience-based		Learning from past interactions and historical experience	Optimus-1 [82], Experiential Co-Learning [120], Expel [211], AutoManual [15], AutoGuide [41], Agent Hospital [76]
Feedback-based	Self-Reflection	Self-evaluating and correcting to refine current behavior	Reflexion [136], QueryAgent [58], SAGE [85], Agent-pro [207], Symbolic Learning [219], NLRL [39]
	External Feedback	Modifying behaviors via external signals and critiques	Retroformer [189], COPPER [8], InteRecAgent [59], CoE [175], MPO [176], WKM [123]
	Meta-Prompt	Iteratively refining global instructions/prompts	MetaReflection [50], Retroformer [189], OPRO [181], Symbolic Learning [219], FinCon [193], Mass [216], MPO [176]
Tool-based		Using external tools and optimizing selection strategies	TPTU [130], AVATAR [167], Lyra [213], Middleware [47], AgentOptimizer [206], VideoAgent [35], AutoAct [124]
Retrieval-based		Retrieving and incorporating external knowledge	AutoRAG [64], Self-RAG [6], RaDA [65], MALADE [22], EMG-RAG [160], WebThinker [79], FlowSearch [56]
Multi-Agent collaboration		Multiple role-specific agents communicate and collaborate	MetaGPT [53], ChatDev [121], DyLAN [97], MacNet [122], Agentverse [17], CAPO [92], SMOA [73], MAD [83], AutoGen [166], Yuksel et al. [196]

agent. Agent Hospital [76] integrates a medical record library and an experience repository to refine guidelines based on successful and failed cases. To extract actionable insights, methods like Expel [211], AutoManual [15], and AutoGuide [41] autonomously collect, organize, and retrieve experiential knowledge to guide future actions during inference. AutoMLGen [33] propose MCGS to obtain more experiential solutions by integrating information from different branches. In addition, Experiential Co-Learning [120] introduces a framework where instructor and assistant agents collaboratively gather shortcut-oriented experiences from their historical trajectories. These experiences are reused to inform future task execution, enabling agents to iteratively improve through shared and accumulated knowledge.

4.2 Feedback-based Optimization

Feedback-based optimization enhances LLM-based agents by incorporating various forms of feedback for self-reflection, correction, and iterative improvement. These approaches operate without modifying model parameters and thus complement fine-tuning and RL by providing lightweight, inference-time mechanisms for adaptive behavior adjustment. We categorize work into three types based on feedback strategies and their role in triggering iterative improvements: feedback-based self-reflection optimization, external feedback-based optimization, and meta-prompt optimization.

4.2.1 Feedback-based Self-Reflection Optimization. This category focuses on agent’s reflection ability using feedback from the environment or self-evaluation. The agent uses feedback to identify areas for improvement and adjust its behavior through self-correction and evolution, enhancing its adaptability and performance in dynamic environments.

Reflexion [136] and QueryAgent [58] leverage task outcomes or heuristic evaluations, converting them into textual feedback to guide self-reflective adjustments. In SAGE [85], the checker agent provides iterative feedback on the current solution, while the assistant agent generates self-reflection to support continual refinement. Agent-pro [207] reflects on past trajectories and beliefs to correct suboptimal strategies. Additionally, Symbolic Learning [219] and NLRL [39] use natural language to mimic optimization processes such as gradient updates or reward shaping, treating these evaluations as feedback that incrementally improves policy execution.

These approaches illustrate how inference-time reflection can emulate aspects of parameter-level optimization while remaining more flexible and interpretable.

4.2.2 External Feedback-based Optimization. In this part, evaluation signals like reflections and corrections from external models or frameworks are applied to refine the behavior of the actor agent. Inspired by Actor-Critic algorithm, these methods enhance the robustness and adaptability of LLM-based agents and are also commonly used in multi-agent collaborative frameworks.

Retroformer [189] and WKM [123] train a retrospection model and a world knowledge model to analyze failures and support global planning respectively, enabling agent to iteratively adjust actions. Similarly, COPPER [8] employs a shared reflector module to generate counterfactual feedback, refine prompts, and store improvements in a memory module. InteRecAgent [59] applies a critic LLM checking the actor agent's behavior to prevent instruction violations or incorrect tool usage. In CoE [175], a multi-agent framework is proposed to address complex operations research, where a conductor orchestrates task assignment among specialized agents, and an evaluator provides feedback to iteratively refine the outputs through backward optimization.

4.2.3 Meta-Prompt Optimization. Meta-prompt optimization refines global instructions or meta-prompts to enhance the generalization capabilities of LLM-based agents. By iteratively adjusting prompts based on feedback, agents can better adapt to diverse and dynamic tasks, optimizing their behaviors from a broader and more general perspective.

MetaReflection [50] and OPRO [181] extract information from failed trials or analyze task accuracy to create optimized prompts, iteratively refining instructions to align agent behavior with task objectives. Mass [216] extends prompt optimization to a hierarchical framework, combining local prompt adjustments, workflow restructuring, and global refinements to support more stable multi-step reasoning. MPO [176] fine-tunes a meta-planner through SFT and DPO to guide high-level planning via meta-prompts, illustrating how prompt-level and parameter-level optimization can work together. Other approaches such as Symbolic Learning [219] and FinCon [193] simulate optimization processes like gradient descent or loss shaping in natural language, enabling agents to iteratively adjust strategies without modifying underlying model parameters. These methods show how meta-prompt optimization can introduce structured behavioral priors that reinforce or extend the capabilities obtained through parameter-based training.

4.3 Tool-based Optimization

One of the key distinctions between LLMs and agents is their ability to utilize external tools. These tools, such as search engines, code interpreters and domain-specific modules, enable agents to perform tasks more autonomously. Through effective selection and utilization of tools, agents can significantly expand their problem-solving scope and capabilities.

TPTU [130] enhances task planning and tool usage by optimizing task decomposition and tool invocation. AVATAR [167] employs a comparator to analyze performance differences between sample pairs, attributing discrepancies to tool usage issues and providing actionable improvements. Middleware [47] introduces error feedback mechanisms to align tool inputs and outputs across steps, mitigating execution errors and improving system robustness. Additionally, AgentOptimizer [206] optimizes tool usage by treating functions as learnable weights, then iteratively refines function sets based on execution outcomes without modifying core LLM parameters. Other methods prioritize efficient tool integration in dynamic environments. For instance, VideoAgent [35] employs a minimal yet sufficient tool set, incorporating specialized visual models and object memory queries. AutoAct [124] automates task trajectory generation using a pre-assembled library of tools, such as search engines and code interpreters.

4.4 Retrieval-based Optimization

Retrieval-based approaches enhance LLM-based agents by integrating external evidence into the reasoning process, enabling models to supplement fixed pre-trained knowledge with dynamically acquired information from heterogeneous sources. By grounding decisions in retrieved context, these methods improve factual accuracy and adaptability in evolving environments. Beyond early retrieval-augmented techniques such as RAG, recent systems like **Deep Research (DR)** [179] further advance this paradigm through multi-step and iterative retrieval and report generation.

Early approaches primarily optimize retrieval configurations or integrate retrieval with generative processes. For example, AutoRAG [64] explores combinations of retrieval, re-ranking, and expansion strategies to identify effective pipelines, while Self-RAG [6] couples retrieval with self-reflection to iteratively refine generated content. Other systems leverage retrieval to guide task planning and execution: RaDA [65] incorporate dynamic retrieval and past experiences to support decomposition and action generation, and PaperQA [68] extends retrieval-augmented techniques to scientific domains by extracting fine-grained evidence from full-text articles. Multi-agent frameworks such as MALADE [22] further apply retrieval to enhance coordinated decision-making, while EMG-RAG [160] learns to select memory via reinforcement learning to provide more informative contextual prompts. Building on these foundations, recent DR agents [133, 179] significantly extend the retrieval paradigm through multi-step search gathering and structured reasoning. WebThinker [79] introduces a deep web exploration workflow that autonomously performs reasoning–search–synthesis cycles to generate research reports, strengthened by RL to improve tool calling capability. FlowSearch [56] develops a dynamic, multi-agent knowledge-flow framework that enables parallel exploration, recursive evidence integration, and adaptive workflow optimization for complex research-oriented tasks.

4.5 Multi-Agent Collaborative Optimization

Multi-agent collaborative optimization enhances LLM-based systems by distributing roles, coordinating information flow, and enabling agents to jointly plan and execute tasks. Through effective collaboration, these frameworks enhance collective decision-making in LLM-based **multi-agent systems (MAS)**, achieving performance beyond the capabilities of individual agents.

Early systems such as MetaGPT [53] and ChatDev [121] formalize human-inspired workflows for software engineering by assigning distinct roles for planning, coding, debugging, and documentation. Recent programming-oriented agents extend this paradigm further. For example, Claude Code provides a general-purpose multi-agent coding framework in which multiple specialized subagents coordinate long-horizon tasks through structured planning, execution, and memory-assisted refinement, enabling AI to operate as an engineering team. To improve coordination and organizational efficiency, DyLAN [97] and MacNet [122] dynamically construct agent networks using importance scoring and early-stopping signals to streamline reasoning. Agentverse [17] and CAPO [92] enable iterative proposal, evaluation, and refinement of plans, supporting complex task decomposition. More recently, Yuksel et al. [196] introduce an autonomous multi-agent refinement loop in which agents collectively propose and validate system-level modifications, enabling end-to-end self-optimization. Debate-oriented frameworks such as SMoA [73] and MAD [83] further enhance decision quality by coordinating multiple agents under a judge or selector.

Although parameter-free optimization methods primarily operate at inference time, recent research increasingly combines them with parameter-based approaches to achieve more stable and higher-capacity agent behavior. A growing body of recent work suggests that effective agent optimization increasingly benefits from combining parameter-based and parameter-free strategies, as each addresses complementary aspects of agent capability. For instance, Retrospect

Table 8. Representative Open-Source Frameworks and Stacks for Building LLM-based Agents

Framework	Core Capabilities	Strengths	Limitations	Code
LangGraph	Modular and graph-based orchestration	Visual workflow design; fine-grained control	Structural complexity; steep learning curve	Link
AutoGen	Multi-agent dialogue coordination	Flexible collaboration; complex problem handling	High debugging cost; resource-intensive	Link
CrewAI	Role-task workflow design	Team-oriented collaboration; lightweight usage	Limited scalability; evolving ecosystem	Link
Camel-AI	Role-driven interaction protocols	Natural role-play; simple agent specification	Narrow task scope; moderate scalability	Link
Semantic Kernel	Lightweight AI orchestration	Strong system integration; high developer efficiency	Platform coupling; less mature ecosystem	Link
LlamaIndex	Data-centric retrieval-oriented pipelines	Strong retrieval modules; rich data connectors	Data dependence; limited autonomous behavior	Link
AgentScope	Scalable multi-agent execution	Engineering-ready design; distributed operation	High resource demand; higher integration overhead	Link

[171] incorporates supervised fine-tuning with value-guided weighting during retrospection, SwiftSage [86] alternates between a fine-tuned model and prompt-enhanced reasoning, and NLRL [39] expresses reinforcement signals in natural language that can subsequently be distilled through fine-tuning. These examples indicate that modern LLM-based agent systems often integrate both paradigms to achieve more reliable and adaptable behavior than either approach alone.

5 Open-source Frameworks and Stacks

5.1 Background and Introduction

LLM-based agents, capable of autonomous perception, reasoning, decision-making, and action, have rapidly become a core paradigm for building complex AI applications. As these agents integrate natural-language interfaces, external tools, and environment interaction, their development requires standardized and reusable infrastructure. To support this need, open-source frameworks such as LangChain¹ and AutoGen [166] offer modular abstractions for planning, memory management, tool invocation, and execution control, substantially reducing engineering complexity and serving as the foundation for many recent agent systems and optimization methods.

Despite differences in design, most LLM agent frameworks share a common set of core components. **Planning** decomposes high-level goals into actionable substeps. **Tool use** enables agents to invoke external APIs or functions to access information or perform operations beyond language generation. **Memory** modules store short-term context and long-term experiential knowledge, allowing decisions to incorporate prior interactions. **Reflection** mechanisms help agents evaluate outputs, identify errors, and refine future actions. Together, these components form a unified architectural foundation for building robust and adaptable LLM-based agents.

5.2 Common Open-source Agent Frameworks

We summarize several widely used and actively maintained open-source frameworks for building LLM-based agents in Table 8. The selection excludes low-code platforms such as Coze,² as well as frameworks that have been discontinued or are no longer actively maintained.

¹<https://github.com/langchain-ai/langchain>

²<https://github.com/coze-dev/coze-studio>

Among these frameworks, orchestration-centered systems such as LangChain and its workflow extension LangGraph emphasize modular composition and graph-structured control, enabling developers to chain LLM calls, integrate external tools, and manage multi-step workflows with fine-grained state tracking. LlamaIndex [91] follows a similar orchestration philosophy but is distinguished by flexible data interfaces and a retrieval-centric design suited for applications that rely on structured or unstructured knowledge sources. In contrast, collaboration-centered frameworks including AutoGen [166], CrewAI [26], Camel-AI [74], and AgentScope [42] focus on role-based agent interaction and coordinated multi-step problem solving. AutoGen provides an actor-style architecture for multi-agent conversation, tool execution, and iterative debugging, whereas CrewAI emphasizes ease of defining team-like roles and workflows. Camel-AI and AgentScope extend this pattern through specialized agent communication and scalable multi-agent environments. Semantic Kernel [148] offers a lightweight, multi-language runtime that integrates AI services into existing software ecosystems while supporting both single-agent and multi-agent patterns. Overall, orchestration-oriented frameworks provide stronger support for workflow representation and tool or data integration, whereas collaboration-oriented frameworks offer greater diversity in multi-agent interaction patterns and role modeling.

6 Evaluation and Datasets for LLM-based Agents

Evaluating LLM-based agents requires both appropriate assessment methodologies and diverse datasets that reflect agent performance in various domains. In this section, we first summarize common categories of agent evaluation approaches, and then review the benchmarks used to measure agent performance as well as the datasets designed for agent tuning and optimization.

6.1 Evaluation Methodologies

Current evaluation methodologies can be broadly grouped into two paradigms [108].

(1) **Human evaluation** remains necessary for tasks that are subjective or lack clear quantitative metrics. It is commonly used for assessing text quality, creativity, coherence, emotional intelligence, or specialized reasoning-scenarios where correctness cannot be fully captured by predefined rules. Human judgments reflect user preferences and provide nuanced assessments but are costly, difficult to scale, and prone to variability, limiting their use in large-scale or iterative agent evaluation.

(2) **Automated evaluation** has become the primary approach for LLM-based agents due to its scalability and reproducibility. **Static dataset evaluation** relies on curated benchmarks and predefined metrics (e.g., accuracy or execution success rate) and works well for objective tasks [108], though it may overlook interactive behaviors. **LLM-as-a-judge methods** extend automation to subjective tasks by using stronger models to score or compare outputs [72], enabling automated assessment for subjective or open-ended tasks. Recent multi-dimensional automated protocols integrate rule-based metrics, execution feedback, and model-based judgments within interactive environments, enabling more holistic assessment of reasoning quality, tool-use reliability, and multi-step decision-making.

6.2 Datasets and Benchmarks for Evaluation

6.2.1 General Domain Evaluation Tasks. We summarize common evaluation datasets categorized by general task domains in Table 9, including programming, **question answering (QA)**, multimodal tasks. Following AgentBank [140], we also use action space to distinguish tasks where actions like natural language and code are classified into continuous space.

Math: Mathematical reasoning tasks assess an agent's ability to perform multi-step inference and solve quantitative problems. Common evaluation metrics include **exact-match (EM)** accuracy and Pass@k, which measure the correctness of final generated answers under deterministic or

Table 9. Summary of Commonly Used Datasets and Environments for Evaluation Tasks

Domain	Datasets/Environments	Task Type	Action Space	Data Link
Math	GSM8K [25]	Mathematical reasoning	Continuous	Link
	AsDIV [107]	Mathematical reasoning	Continuous	Link
	SVAMP [116]	Mathematical reasoning	Continuous	Link
	MATH [52]	Mathematical reasoning	Continuous	Link
	AIME	Mathematical reasoning	Continuous	Link
QA	HotpotQA [186]	Open-domain QA	Continuous	Link
	MMLU [51]	Multiple choice QA	Continuous	Link
	TruthfulQA [88]	Multiple choice QA, Open-domain QA	Continuous	Link
	PubMedQA [62]	Domain-Specific QA	Continuous	Link
	MuSiQue [149]	Commonsense Reasoning	Continuous	Link
	QASPER [27]	Domain-Specific QA	Continuous	Link
Code	ARC [23]	Commonsense Reasoning	Continuous	Link
	SWE-bench [183]	Code Generation, Bug Fixing	Continuous	Link
	HumanEval [16]	Code Generation	Continuous	Link
	LiveCodeBench [110]	Code Execution, Bug Fixing	Continuous	Link
	BIRD [75]	Text-to-SQL Conversion	Continuous	Link
	InterCodeSQL [184]	Code Generation	Continuous	Link
Tool-use	MLE-Bench [12]	algorithm design	Continuous	Link
	T-Eval [19]	Tool-use	Continuous	Link
	MINT-Bench [159]	Tool-use	Continuous	Link
	ToolEval [125]	Tool-use	Continuous	Link
	MTU-Bench [156]	Tool-use	Continuous	Link
Web	WebShop [187]	Web Navigation	Discrete	Link
	WebArena [218]	Web Navigation	Discrete	Link
	Mind2Web [28]	Web Navigation	Discrete	Link
	MiniWoB++ [89]	Web Navigation	Discrete	Link
Environment Interaction	ScienceWorld [157]	Scientific Experiment	Discrete	Link
	ALFWorld [138]	Embodied AI	Discrete	Link
	TDW-MAT [201]	Embodied AI	Discrete	Link
	C-WAH [201]	Embodied AI	Discrete	Link
	ALFRED [137]	Embodied AI	Discrete	Link
	RLCard [199]	Game	Discrete	Link
	Breakthrough [69]	Game	Discrete	Link
Multimodal	VQA v2.0 [5]	Visual QA	Continuous	Link
	A-OKVQA [132]	Visual QA	Continuous	Link
	ScienceQA (IMG) [98]	Visual QA	Continuous	Link
	EgoSchema [104]	Video Understanding, Visual QA	Continuous	Link
	NEXT-QA [172]	Video Understanding, Visual QA	Continuous	Link

sampled decoding. Representative datasets include GSM8K [25] for arithmetic reasoning, SVAMP [116] for robustness to linguistic variations, and MATH [52] or AIME for evaluating advanced competition-level problem-solving.

QA: Question answering tasks assess an agent’s ability to comprehend text, retrieve relevant information, and perform single- or multi-step reasoning to produce accurate answers. Common evaluation metrics include EM and F1 score, which measure correctness and overlap with gold-standard answers; multi-choice datasets typically use accuracy. Representative benchmarks include HotpotQA [186] and MuSiQue [149] for evaluating multi-hop reasoning across documents, and MMLU [51] for broad knowledge coverage across diverse academic subjects. Domain-specific

datasets such as PubMedQA [62] assess reasoning over biomedical literature, while ARC [23] focuses on scientific reasoning at the grade-school level.

Code: Code-grounded tasks evaluate an agent’s ability to understand specifications, generate executable programs, fix bugs, and reason over large codebases. Common evaluation metrics include pass@k for function-level code synthesis, execution success rate for verifying correctness against test cases, and runtime or efficiency of programs. For end-to-end algorithmic challenges such as MLE-Bench [12], evaluation focuses on machine learning task performance (e.g., acc, loss), reflecting an agent’s capability in algorithm design. HumanEval [16] provides a standard setting for Python function synthesis, while SWE-bench [183] targets real-world software engineering tasks that require generating repository-grounded patches. LiveCodeBench [110] extends evaluation to competitive programming problems, and database-oriented benchmarks such as BIRD [75] and InterCodeSQL [184] measure SQL generation and refinement through execution feedback.

Tool-use: Tool-use tasks assess an agent’s ability to plan and execute interactions with external tools or APIs, requiring accurate tool selection, parameter construction, and integration of tool outputs into multi-step reasoning. Common evaluation metrics include tool selection accuracy, tool-call success rate, and tool amount or order accuracy in multi-turn tool-use settings. Representative datasets include T-Eval [19] for fine-grained diagnostics of planning and tool invocation, ToolEval [125] for automated functional scoring, and MTU-Bench [156] for multi-granularity scenarios with prediction-based metrics such as tool selection and parameter accuracy. MINT-Bench [159] further evaluates multi-turn tool interactions driven by natural-language feedback.

Web: Web-based tasks evaluate an agent’s ability to navigate websites, understand structured and unstructured content, and complete goal-directed interactions. Common evaluation metrics include task success rate, navigation efficiency (e.g., number of steps), and action accuracy for page-level operations. Representative benchmarks include WebShop [187] for instruction-following in e-commerce settings and WebArena [218] for interacting with full-featured web applications. Larger-scale datasets such as Mind2Web [28] and MiniWoB++ [89] further provide diverse real-world or synthetic environments for evaluating multi-step web navigation behaviors.

Environment Interaction: Environment interaction tasks evaluate an agent’s ability to perform multi-step decision-making and adapt to feedback within dynamic physical or virtual environments. Common evaluation metrics include task success rate, average score, and reward or win rate in game-based settings. Representative benchmarks include ScienceWorld [157] for text-based scientific experimentation and embodied environments such as ALFWorld [138] and ALFRED [137], which require language-guided object manipulation and navigation. Game-oriented platforms like RLCard [199] and OpenSpiel [69] further assess strategic planning and multi-agent decision-making across competitive and cooperative tasks.

Multimodal: In multimodal tasks, agents integrate information across images, text, and videos. Common evaluation metrics include answer accuracy, VQA accuracy, multiple-choice accuracy for image–text reasoning, and temporal reasoning accuracy for video-based tasks. VQA-V2 [5] tests core visual question answering abilities, while A-OKVQA [132] and ScienceQA (IMG) [98] introduce knowledge-intensive visual reasoning challenges. Video benchmarks [104, 172] further assess temporal, causal, and activity-level understanding in dynamic visual scenes.

6.2.2 Multi-task Benchmarks. To evaluate agents’ ability to generalize across diverse domains, multi-task benchmarks integrate heterogeneous task types into unified evaluation suites, providing a comprehensive assessment of performance. Datasets such as AgentBench and AgentEval focus on programming, web navigation, and tool-augmented decision-making, often using unified formats such as ReAct to consolidate reasoning and action traces. Benchmarks including Just-Eval, StreamBench, and AgentBoard extend evaluation to broader scenarios—text reasoning, embodied interaction, and game environments—aggregating diverse settings to assess generalization across

Table 10. Multi-Task Agent Evaluation Benchmarks

Benchmark	# Datasets	# Task Types	Test Size	Covered Domains	Data Link
AgentBench [95]	8	3	1,091	③ ⑤ ⑦	Link
AgentEval [169]	14	5	1,160	③ ④ ⑤ ⑥ ⑦	Link
Just-Eval [87]	9	7	1,000	① ② ⑤ ⑧ ⑨	Link
StreamBench [164]	7	5	9,702	① ⑤ ⑥	Link
AgentBoard [100]	9	4	1,013	③ ④ ⑥ ⑦	Link
GAIA [106]	-	5	466	① ③ ⑤ ⑥	Link

We use ①, ②, ③, and so on., to represent the covered task domains: ① QA, ② Math, ③ Web, ④ Embodied AI, ⑤ Code, ⑥ Tools, ⑦ Game, ⑧ Writing, ⑨ Role-playing.

Table 11. Datasets for Agent Tuning

Dataset	# Tasks	# Filtered Trajectories	Covered Domains	Data Link
AgentInstruct [197]	6	1,866	③ ④ ⑤	Link
AgentBank [140]	16	51,287	① ② ③ ④ ⑤	Link
Agent-FLAN [21]	7	24,703	③ ⑤ ⑥	Link
AgentOhana [202]	10	42,600	③ ⑤ ⑥	Link
FireAct [13]	3	1,344	①	Link
ToRA-CORPUS [46]	2	15,538	②	Link
AgentTraj/AgentTraj-L [169]	14	6,130/14,485	③ ④ ⑤ ⑥ ⑦	Link
SMART-Trajectory [195]	17	142,507	① ⑧ ⑨ ⑩	Link

We use ①, ②, ③, and so on., to represent the task domains as follows: ① QA (Commonsense Reasoning and Open-domain QA), ② Math, ③ Web, ④ Embodied AI, ⑤ Code, ⑥ Tools, ⑦ Game, ⑧ Fact Verification, ⑨ Dialogue, ⑩ General Instructions.

varied modalities and task structures. GAIA introduces a realistic assistant-oriented paradigm requiring reasoning, multimodal understanding, web interaction, and tool usage, offering a rigorous benchmark for progress toward general-purpose AI assistants. Table 10 summarizes representative benchmarks, including their coverage, domain types, sample sizes, and resource links.

6.3 Datasets for Agent Tuning

Fine-tuning datasets provide trajectories that equip LLM-based agents with task-specific behaviors, enabling them to acquire skills such as tool use, web interaction, programming, mathematical reasoning, and embodied decision-making. Recent tuning corpora such as AgentInstruct, AgentBank, Agent-FLAN, AgentOhana, FireAct, ToRA-CORPUS, AgentTraj, and SMART-Trajectory—integrate trajectories collected or synthesized from diverse sources including web environments, embodied platforms, SQL querying tasks, scientific reasoning datasets, and multi-turn ReAct-style interactions. These datasets typically combine curated demonstrations with filtered trajectories from large models, standardize multi-turn formats, and cover heterogeneous action spaces, thereby supporting scalable behavioral cloning and instruction-following fine-tuning across a wide range of agent scenarios. Table 11 summarizes their task coverage, trajectory volume, and data sources.

7 Application

LLM-based agents have been applied across diverse domains, showing their potential to address complex tasks and enhance productivity. This section provides an overview of representative applications in Healthcare, Science, Embodied Intelligence, and Finance, as illustrated in Figure 4.

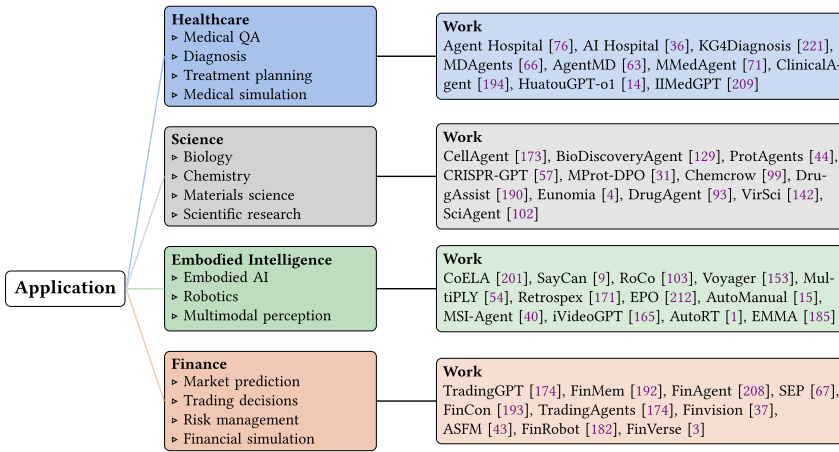


Fig. 4. The applications of LLM-based agents.

7.1 Healthcare

LLM-based agents in healthcare generally follow a progressive workflow that spans medical question answering, diagnosis, treatment planning, and medical simulation, with early systems enhancing conversational accuracy through domain-specific fine-tuning [7, 139]. Subsequent work extends these capabilities through simulation frameworks [36, 76], which provide dynamic clinical environments for realistic physician–patient interactions, and through multi-agent systems such as KG4Diagnosis [221] and MDAgents [66], which model collaboration between general practitioners and specialists to improve diagnostic reasoning. More recent methods [14, 209] further combine fine-tuning with RL techniques such as PPO or DPO to strengthen long-horizon clinical reasoning and treatment optimization. Despite these advances, key challenges persist, including the risks posed by medical hallucinations, strict privacy and compliance constraints, and the limited context capacity of LLMs for processing full electronic medical records.

7.2 Science

LLM-based agents are increasingly used to support scientific research by integrating domain knowledge retrieval, hypothesis generation, and automated experiment design into unified workflows that accelerate biological and chemical discovery. Recent work in biology, such as CellAgent [173] and BioDiscoveryAgent [129], shows how multi-agent coordination and iterative self-optimization can automate tasks like single-cell data analysis and gene perturbation studies. Tool-integrated agents including ProtAgents [44] and CRISPR-GPT [57] further leverage external scientific software to assist protein design and gene editing. Emerging frameworks now aim at designing more general AI scientist agents. AlphaEvolve [111] illustrates how code-evolving agents can autonomously discover more efficient algorithms and scientific constructs, while InternAgent [147] introduces a closed-loop multi-agent system spanning idea generation, methodology design, and experimental execution. However, scientific workflows remain challenging due to high computational costs, long knowledge-retrieval cycles, and the complexity of coordinating multi-stage experimental pipelines, highlighting the need for more efficient and scalable scientific agent systems.

7.3 Embodied Intelligence

In the fields of embodied intelligence and robotic systems, LLM-based agents autonomously interact with the physical world, and integrate multimodal inputs such as visual and auditory inputs to

perform complex tasks. Early systems demonstrate how perception, memory, and planning modules can be combined to execute complex embodied tasks [103, 201], while RL-based approaches such as SayCan [9] ground LLM-driven reasoning in real-world robotic actions. In interactive simulation settings, agents like Voyager [153] illustrate how long-term memory and iterative skill acquisition support continual learning. More recent frameworks [15, 40, 54, 171, 212] refine action strategies through environmental feedback and structured sub-goal decomposition, and multimodal methods [1, 165, 185] integrate visual, textual, and action signals to improve task planning and execution. Despite these advances, embodied systems face persistent challenges, including uncertainty in real-world environments, safety and reliability concerns for physical actuation, and the difficulty of maintaining consistent performance over long-horizon tasks.

7.4 Finance

LLM-based agents are increasingly adopted in financial applications ranging from market prediction and trading decisions to risk management and financial simulation, where they process heterogeneous financial signals and support data-driven reasoning. A variety of agent architectures have been explored: models equipped with hierarchical memory and reflective evaluation [81, 192, 208] aim at capturing shifting market patterns, and trading agents that incorporate finance signals such as SEP [67] adapt their strategies through iterative policy refinement. MAS [37, 174, 193] extend financial analysis by coordinating diverse roles, enabling analyst-style reasoning, risk monitoring, and collaborative planning. Meanwhile, cross-modal frameworks like ASFM [43] and FinRobot [182] fuse textual, numerical, and visual financial information to support decisions in complex market settings, and systems such as FinVerse [3] augment analysis with code-execution capabilities for fine-grained financial computation. Applying these agents in practice requires careful consideration of financial data sensitivity, the fast-changing nature of market knowledge, and the demand for transparent and trustworthy decision-making in high-stakes environments.

8 Challenges and Future Directions

We outline the key challenges faced in current state of LLM-based agent optimization and explore potential future research directions.

8.1 Algorithm Adaptability and Efficiency

A key problem in LLM-based agent optimization is balancing algorithmic efficiency with task-specific adaptability. Current methods, such as RL and fine-tuning, often struggle with sparse rewards, large action spaces, or overfitting to specific data. Algorithms like PPO are effective for policy optimization but computationally expensive, while DPO simplifies training but is primarily suited for single-step optimization, limiting its effectiveness for multi-step interactive tasks. Moreover, many existing methods fine-tune LLM-based agents using agent trajectory data for specific domains, which results in poor performance on unseen domains, particularly when there is a significant mismatch between training and real-world data distributions.

Exploring hybrid approaches that combine RL and fine-tuning offers a potential pathway to improving both adaptability and efficiency. Optimizing reward design and integrating efficiency-focused algorithms will also be crucial for reducing computational costs while maintaining high adaptability. Additionally, techniques like distribution alignment and domain adaptation can enable agents to transfer knowledge more effectively between tasks, even in dynamic environments.

8.2 Standardized Evaluation Metrics

The evaluation of LLM-based agents currently lacks standardized metrics, making fair performance comparison across tasks challenging. Different agents operate in varied environments, such as

mathematical reasoning, web navigation, and embodied AI, each relying on distinct evaluation criteria. This variability makes it difficult to compare optimization methods fairly and assess their generalizability. Additionally, existing metrics primarily assess task completion rather than extent of optimization, making it hard to quantify the effectiveness of different enhancement techniques.

Establishing standardized evaluation frameworks provides a promising direction for facilitating fair comparisons across diverse LLM-based agent tasks, such as developing unified benchmarks that assess optimization effectiveness beyond task completion. Furthermore, creating proper metrics that track stepwise optimization progress and integrating preference-based evaluation signals could provide a more comprehensive assessment.

8.3 Cost and Efficiency Constraints in LLM-based Agents

LLM-based agents face substantial cost and efficiency constraints when deployed in long-horizon or tool-augmented settings. Achieving strong performance frequently depends on large-scale or proprietary reasoning models, whose API usage and on-premise deployment impose significant financial and computational overhead. Multi-turn interaction and extended reasoning chains further amplify latency and token consumption, making many real-world workflows difficult to scale. Although advanced reasoning models can enhance decision quality, their integration typically increases inference depth and cumulative token cost, limiting the practicality of deploying LLM-based agents in demanding operational environments.

Efforts to alleviate these constraints focus on reducing redundant inference and lowering the cost of environment construction. Techniques such as context optimization [204], adaptive memory mechanisms, and inference-efficient prompt engineering offer promising pathways for limiting token usage in multi-step interactions. In parallel, scalable training environments—particularly those that can be automatically generated or locally deployed—help reduce development overhead and enable more cost-effective agent training. Advancing these directions will be critical for making LLM-based agents viable at scale in long-horizon and resource-intensive scenarios.

8.4 Safety and Robustness of Autonomous Agents

LLM-based agents are becoming increasingly capable across planning, tool interaction, and multi-step decision-making. However, their expanded autonomy also exposes them to safety risks during real-world deployment. Agents remain vulnerable to prompt injection and jailbreak attacks, where crafted inputs can override intended behaviors or bypass system constraints. In tool use setting, incorrect or manipulated intermediate outputs may propagate into external actions, leading to unintended or unsafe executions. Moreover, when operating across diverse or shifting environments, agents often exhibit limited generalization, resulting in inconsistent or unpredictable behavior.

Mitigating these risks requires defense mechanisms at both system and model levels. Recent work [198] introduces multi-agent defense frameworks to filter harmful responses and counter jailbreak-style prompts across models. System-level strategies including input-output filtering, fact-checking pipelines, multi-model consensus, and human-in-the-loop validation help constrain unsafe behaviors in multi-step interactions. At the model level, alignment-driven training and safety-aware optimization remain essential for producing reliable and policy-compliant outputs.

8.5 LLM-based Multi-Agent System Optimization

LLM-based multi-agent optimization remains underexplored, with most approaches improving single agent while leaving the collective behavior of the system largely unaddressed. Existing multi-agent strategies typically rely on frozen LLMs and prompt-based workflow, limiting joint optimization and leading to inconsistent communication and conflicting objectives. Recent empirical studies, such as MAST [11], further show that many MAS failures stem not from model

limitations but from structural issues—insufficient role specification, inter-agent misalignment, and weak validation or termination mechanisms. These observations highlight that advancing multi-agent optimization requires both parameter-level improvements and more reliable coordination structures.

Developing robust multi-agent parameter optimization methods is essential for agent systems [195]. Joint parameter tuning and effective information exchange should be further explored to align agent objectives and maximize system performance. Moreover, reward-sharing mechanisms and hierarchical decision-making could further enable LLM-based MAS to operate effectively in dynamic and complex environments.

9 Conclusion

This survey provides a comprehensive overview of the optimization methods for LLM-based agents, categorizing them into parameter-driven and parameter-free approaches. We focus on parameter-driven optimization, encompassing conventional fine-tuning, RL-based methods, and hybrid strategies. We first detail the entire workflow from data construction to fine-tuning in the conventional fine-tuning part. Subsequently, we outline RL-based optimization, covering both reward function optimization and preference alignment methods. Then, we introduce parameter-free approaches, including historical experience integration, feedback mechanisms, tool usage, retrieval, and multi-agent collaboration. Furthermore, we summarize datasets widely utilized to evaluate or fine-tune agents and introduce the diverse real-world applications of LLM-based agents. Finally, we identify several key challenges and propose future research directions aimed at driving the development of more capable and intelligent LLM-based agents.

References

- [1] Michael Ahn, Debidatta Dwibedi, Chelsea Finn, Montse Gonzalez Arenas, Keerthana Gopalakrishnan, Karol Hausman, Brian Ichter, Alex Irpan, Nikhil Joshi, Ryan Julian, et al. 2024. Autort: Embodied foundation models for large scale orchestration of robotic agents. arXiv:2401.12963. Retrieved from <https://arxiv.org/abs/2401.12963>
- [2] Renat Aksitov, Sobhan Miryoosefi, Zonglin Li, Daliang Li, Sheila Babayan, Kavya Kopparapu, Zachary Fisher, Ruiqi Guo, Sushant Prakash, Pranesh Srinivasan, et al. 2023. Rest meets react: Self-improvement for multi-step reasoning llm agent. arXiv:2312.10003. Retrieved from <https://arxiv.org/abs/2312.10003>
- [3] Siyu An, Qin Li, Junru Lu, Di Yin, and Xing Sun. 2024. FinVerse: An autonomous agent system for versatile financial analysis. arXiv:2406.06379. Retrieved from <https://arxiv.org/abs/2406.06379>
- [4] Mehrad Ansari and Seyed Mohamad Moosavi. 2024. Agent-based learning of materials datasets from the scientific literature. *Digital Discovery* 3, 12 (2024), 2607–2617.
- [5] Stanislaw Antol, Aishwarya Agrawal, Jiasen Lu, Margaret Mitchell, Dhruv Batra, C. Lawrence Zitnick, and Devi Parikh. 2015. Vqa: Visual question answering. In *Proceedings of the IEEE International Conference on Computer Vision*. 2425–2433.
- [6] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2024. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *Proceedings of the 12th International Conference on Learning Representations*.
- [7] Zhijie Bao, Wei Chen, Shengze Xiao, Kuang Ren, Jiaao Wu, Cheng Zhong, Jiajie Peng, Xuanjing Huang, and Zhongyu Wei. 2023. Disc-medllm: Bridging general large language models and real-world medical consultation. arXiv:2308.14346. Retrieved from <https://arxiv.org/abs/2308.14346>
- [8] Xiaohu Bo, Zeyu Zhang, Quanyu Dai, Xueyang Feng, Lei Wang, Rui Li, Xu Chen, and Ji-Rong Wen. 2025. Reflective multi-agent collaboration based on large language models. *Advances in Neural Information Processing Systems* 37 (2025), 138595–138631.
- [9] Anthony Brohan, Yevgen Chebotar, Chelsea Finn, Karol Hausman, Alexander Herzog, Daniel Ho, Julian Ibarz, Alex Irpan, Eric Jang, Ryan Julian, et al. 2023. Do as i can, not as i say: Grounding language in robotic affordances. In *Proceedings of the Conference on Robot Learning*. PMLR, 287–318.
- [10] Lorenzo Canese, Gian Carlo Cardarilli, Luca Di Nunzio, Rocco Fazzolari, Daniele Giardino, Marco Re, and Sergio Spanò. 2021. Multi-agent reinforcement learning: A review of challenges and applications. *Applied Sciences* 11, 11 (2021), 4948.

- [11] Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, et al. 2025. Why do multi-agent llm systems fail? arXiv:2503.13657. Retrieved from <https://arxiv.org/abs/2503.13657>
- [12] Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, et al. 2024. Mle-bench: Evaluating machine learning agents on machine learning engineering. arXiv:2410.07095. Retrieved from <https://arxiv.org/abs/2410.07095>
- [13] Baian Chen, Chang Shu, Ehsan Shareghi, Nigel Collier, Karthik Narasimhan, and Shunyu Yao. 2023. Fireact: Toward language agent fine-tuning. arXiv:2310.05915. Retrieved from <https://arxiv.org/abs/2310.05915>
- [14] Junying Chen, Zhenyang Cai, Ke Ji, Xidong Wang, Wanlong Liu, Rongsheng Wang, Jianye Hou, and Benyou Wang. 2024. Huatuogpt-01, towards medical complex reasoning with llms. arXiv:2412.18925. Retrieved from <https://arxiv.org/abs/2412.18925>
- [15] Minghao Chen, Yihang Li, Yanting Yang, Shiyu Yu, Binbin Lin, and Xiaofei He. 2024. AutoManual: Constructing instruction manuals by LLM agents via interactive environmental learning. In *Proceedings of the Advances in Neural Information Processing Systems*. 589–631.
- [16] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. arXiv:2107.03374. Retrieved from <https://arxiv.org/abs/2107.03374>
- [17] Weize Chen, Yusheng Su, Jingwei Zuo, Cheng Yang, Chenfei Yuan, Chi-Min Chan, Heyang Yu, Yaxi Lu, Yi-Hsin Hung, Chen Qian, et al. 2024. AgentVerse: Facilitating multi-agent collaboration and exploring emergent behaviors. In *Proceedings of the ICLR*.
- [18] Weize Chen, Jiarui Yuan, Chen Qian, Cheng Yang, Zhiyuan Liu, and Maosong Sun. 2024. Optima: Optimizing effectiveness and efficiency for llm-based multi-agent system. arXiv:2410.08115. Retrieved from <https://arxiv.org/abs/2410.08115>
- [19] Zehui Chen, Weihua Du, Wenwei Zhang, Kuikun Liu, Jiangning Liu, Miao Zheng, Jingming Zhuo, Songyang Zhang, Dahua Lin, Kai Chen, et al. 2024. T-eval: Evaluating the tool utilization capability of large language models step by step. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 9510–9529.
- [20] Zhixun Chen, Ming Li, Yuxuan Huang, Yali Du, Meng Fang, and Tianyi Zhou. 2025. ATLaS: Agent tuning via learning critical steps. arXiv:2503.02197. Retrieved from <https://arxiv.org/abs/2503.02197>
- [21] Zehui Chen, Kuikun Liu, Qiuchen Wang, Wenwei Zhang, Jiangning Liu, Dahua Lin, Kai Chen, and Feng Zhao. 2024. Agent-FLAN: Designing data and methods of effective agent tuning for large language models. arXiv:2403.12881. Retrieved from <https://arxiv.org/abs/2403.12881>
- [22] Jihye Choi, Nils Palumbo, Prasad Chalasani, Matthew M. Engelhard, Somesh Jha, Anivarya Kumar, and David Page. 2024. MALADE: Orchestration of LLM-powered agents with retrieval augmented generation for pharmacovigilance. In *Proceedings of the Machine Learning for Healthcare Conference*. PMLR.
- [23] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. Think you have solved question answering? try arc, the ai2 reasoning challenge. arXiv:1803.05457. Retrieved from <https://arxiv.org/abs/1803.05457>
- [24] Jesse Clifton and Eric Laber. 2020. Q-learning: Theory and applications. *Annual Review of Statistics and Its Application* 7, 1 (2020), 279–301.
- [25] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. 2021. Training verifiers to solve math word problems. arXiv:2110.14168. Retrieved from <https://arxiv.org/abs/2110.14168>
- [26] crewAI Team. 2025. crewAI. Retrieved December 2, 2025 from <https://github.com/crewAIInc/crewAI>
- [27] Pradeep Dasigi, Kyle Lo, Iz Beltagy, Arman Cohan, Noah A. Smith, and Matt Gardner. 2021. A dataset of information-seeking questions and answers anchored in research papers. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. 4599–4610.
- [28] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Sam Stevens, Boshi Wang, Huan Sun, and Yu Su. 2023. Mind2web: Towards a generalist agent for the web. *Advances in Neural Information Processing Systems* 36 (2023), 28091–28114.
- [29] Zhirui Deng, Zhicheng Dou, Yutao Zhu, Ji-Rong Wen, Ruibin Xiong, Mang Wang, and Weipeng Chen. 2024. From novice to expert: LLM agent policy optimization via step-wise reinforcement learning. arXiv:2411.03817. Retrieved from <https://arxiv.org/abs/2411.03817>
- [30] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. Qlora: Efficient finetuning of quantized llms. *Advances in Neural Information Processing Systems* 36 (2023), 10088–10115.
- [31] Gautham Dharuman, Kyle Hippe, Alexander Brace, Sam Foreman, Väinö Hatanpää, Varuni K Sastry, Huihuo Zheng, Logan Ward, Servesh Muralidharan, Archit Vasan, et al. 2024. MProt-DPO: Breaking the ExaFLOPS barrier for multimodal protein design workflows with direct preference optimization. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–13.

- [32] Zi-Yi Dou, Cheng-Fu Yang, Xueqing Wu, Kai-Wei Chang, and Nanyun Peng. 2024. Re-rest: Reflection-reinforced self-training for language agents. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 15394–15411.
- [33] Shangheng Du, Xiangchao Yan, Dengyang Jiang, Jiakang Yuan, Yusong Hu, Xin Li, Liang He, Bo Zhang, and Lei Bai. 2025. AutoMLGen: Navigating fine-grained optimization for coding agents. arXiv:2510.08511. Retrieved from <https://arxiv.org/abs/2510.08511>
- [34] Damien Ernst and Arthur Louette. 2024. Introduction to reinforcement learning. *Feuerriegel, S., Hartmann, J., Janiesch, C., and Zschech, P* (2024), 111–126.
- [35] Yue Fan, Xiaojian Ma, Rujie Wu, Yuntao Du, Jiaqi Li, Zhi Gao, and Qing Li. 2024. Videoagent: A memory-augmented multimodal agent for video understanding. In *Proceedings of the European Conference on Computer Vision*. Springer, 75–92.
- [36] Zhihao Fan, Lai Wei, Jialong Tang, Wei Chen, Wang Siyuan, Zhongyu Wei, and Fei Huang. 2025. AI hospital: Benchmarking large language models in a multi-agent medical interaction simulator. In *Proceedings of the 31st International Conference on Computational Linguistics*. 10183–10213.
- [37] Sorouralsadat Fatemi and Yuheng Hu. 2024. FinVision: A multi-agent framework for stock market prediction. In *Proceedings of the 5th ACM International Conference on AI in Finance*. 582–590.
- [38] Peiyuan Feng, Yichen He, Guanhua Huang, Yuan Lin, Hanchong Zhang, Yuchen Zhang, and Hang Li. 2024. AGILE: A novel reinforcement learning framework of LLM agents. In *Proceedings of the 38th Annual Conference on Neural Information Processing Systems*.
- [39] Xidong Feng, Ziyu Wan, Haotian Fu, Bo Liu, Mengyue Yang, Girish A. Koushik, Zhiyuan Hu, Ying Wen, and Jun Wang. 2024. Natural language reinforcement learning. arXiv:2411.14251. Retrieved from <https://arxiv.org/abs/2411.14251>
- [40] Dayuan Fu, Biqing Qi, Yihuai Gao, Che Jiang, Guanting Dong, and Bowen Zhou. 2024. MSI-Agent: Incorporating multi-scale insight into embodied agents for superior planning and decision-making. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 643–659.
- [41] Yao Fu, Dong-Ki Kim, Jaekyeom Kim, Sungryull Sohn, Lajanugen Logeswaran, Kyunghoon Bae, and Honglak Lee. 2024. Autoguide: Automated generation and selection of context-aware guidelines for large language model agents. In *Proceedings of the 38th Annual Conference on Neural Information Processing Systems*.
- [42] Dawei Gao, Zitao Li, Xuchen Pan, Weirui Kuang, Zhijian Ma, Bingchen Qian, Fei Wei, Wenhao Zhang, Yuexiang Xie, Daoyuan Chen, et al. 2024. Agentscope: A flexible yet robust multi-agent platform. arXiv:2402.14034. Retrieved from <https://arxiv.org/abs/2402.14034>
- [43] Shen Gao, Yuntao Wen, Minghang Zhu, Jianing Wei, Yuhang Cheng, Qunzi Zhang, and Shuo Shang. 2024. Simulating financial market via large language model based agents. arXiv:2406.19966. Retrieved from <https://arxiv.org/abs/2406.19966>
- [44] Alireza Ghafarollahi and Markus J. Buehler. 2024. ProtAgents: Protein discovery via large language model multi-agent collaborations combining physics and machine learning. *Digital Discovery* 3, 7 (2024), 1389–1409.
- [45] Dibya Ghosh, Marlos C. Machado, and Nicolas Le Roux. 2020. An operator view of policy gradient methods. *Advances in Neural Information Processing Systems* 33 (2020), 3397–3406.
- [46] Zhibin Gou, Zhihong Shao, Yeyun Gong, yelong shen, Yujiu Yang, Minlie Huang, Nan Duan, and Weizhu Chen. 2024. ToRA: A tool-integrated reasoning agent for mathematical problem solving. In *Proceedings of the 12th International Conference on Learning Representations*.
- [47] Yu Gu, Yiheng Shu, Hao Yu, Xiao Liu, Yuxiao Dong, Jie Tang, Jayanth Srinivasa, Hugo Latapie, and Yu Su. 2024. Middleware for LLMs: Tools are instrumental for language agents in complex environments. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 7646–7663.
- [48] Jian Guan, Wei Wu, zujie wen, Peng Xu, Hongning Wang, and Minlie Huang. 2024. AMOR: A recipe for building adaptable modular knowledge agents through process feedback. In *Proceedings of the Advances in Neural Information Processing Systems*. 126118–126148.
- [49] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. arXiv:2501.12948. Retrieved from <https://arxiv.org/abs/2501.12948>
- [50] Priyanshu Gupta, Shashank Kirtania, Ananya Singha, Sumit Gulwani, Arjun Radhakrishna, Gustavo Soares, and Sherry Shi. 2024. MetaReflection: Learning instructions for language agents using past reflections. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 8369–8385.
- [51] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2021. Measuring massive multitask language understanding. *Proceedings of the International Conference on Learning Representations* (2021).
- [52] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring mathematical problem solving with the MATH dataset. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1, NeurIPS Datasets and Benchmarks*.

- [53] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. 2024. MetaGPT: Meta programming for a multi-agent collaborative framework. In *Proceedings of the ICLR*.
- [54] Yining Hong, Zishuo Zheng, Peihao Chen, Yian Wang, Junyan Li, and Chuang Gan. 2024. Multiply: A multisensory object-centric embodied large language model in 3d world. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 26406–26416.
- [55] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-rank adaptation of large language models. In *Proceedings of the International Conference on Learning Representations*.
- [56] Yusong Hu, Runmin Ma, Yue Fan, Jinxin Shi, Zongsheng Cao, Yuhao Zhou, Jiakang Yuan, Xiangchao Yan, Wenlong Zhang, Lei Bai, et al. 2025. FlowSearch: Advancing deep research with dynamic structured knowledge flow. arXiv:2510.08521. Retrieved from <https://arxiv.org/abs/2510.08521>
- [57] Kaixuan Huang, Yuanhao Qu, Henry Cousins, William A Johnson, Di Yin, Mihir Shah, Denny Zhou, Russ Altman, Mengdi Wang, and Le Cong. 2024. Crispr-gpt: An llm agent for automated design of gene-editing experiments. arXiv:2404.18021. Retrieved from <https://arxiv.org/abs/2404.18021>
- [58] Xiang Huang, Sitao Cheng, Shanshan Huang, Jiayu Shen, Yong Xu, Chaoyun Zhang, and Yuzhong Qu. 2024. QueryAgent: A reliable and efficient reasoning framework with environmental feedback based self-correction. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 5014–5035.
- [59] Xu Huang, Jianxun Lian, Yuxuan Lei, Jing Yao, Defu Lian, and Xing Xie. 2023. Recommender ai agent: Integrating large language models for interactive recommendations. arXiv:2308.16505. Retrieved from <https://arxiv.org/abs/2308.16505>
- [60] Xu Huang, Weiwen Liu, Xiaolong Chen, Xingmei Wang, Hao Wang, Defu Lian, Yasheng Wang, Ruiming Tang, and Enhong Chen. 2024. Understanding the planning of LLM agents: A survey. arXiv:2402.02716. Retrieved from <https://arxiv.org/abs/2402.02716>
- [61] Haolin Jin, Linghan Huang, Haipeng Cai, Jun Yan, Bo Li, and Huaming Chen. 2024. From llms to llm-based agents for software engineering: A survey of current, challenges and future. arXiv:2408.02479. Retrieved from <https://arxiv.org/abs/2408.02479>
- [62] Qiao Jin, Bhuwan Dhingra, Zhengping Liu, William Cohen, and Xinghua Lu. 2019. PubMedQA: A dataset for biomedical research question answering. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. 2567–2577.
- [63] Qiao Jin, Zhizheng Wang, Yifan Yang, Qingqing Zhu, Donald Wright, Thomas Huang, W. John Wilbur, Zhe He, Andrew Taylor, Qingyu Chen, et al. 2024. Agentmd: Empowering language agents for risk prediction with large-scale clinical tool learning. arXiv:2402.13225. Retrieved from <https://arxiv.org/abs/2402.13225>
- [64] Dongkyu Kim, Byoungwook Kim, Donggeon Han, and Matouš Eibich. 2024. AutoRAG: Automated framework for optimization of retrieval augmented generation pipeline. arXiv:2410.20878. Retrieved from <https://arxiv.org/abs/2410.20878>
- [65] Minsoo Kim, Victor Bursztyn, Eunye Koh, Shunan Guo, and Seung-won Hwang. 2024. Rada: Retrieval-augmented web agent planning with llms. In *Findings of the Association for Computational Linguistics ACL 2024*. 13511–13525.
- [66] Yubin Kim, Chanwoo Park, Hyewon Jeong, Yik Siu Chan, Xuhai Xu, Daniel McDuff, Hyeonhoon Lee, Marzyeh Ghassemi, Cynthia Breazeal, Hae Park, et al. 2025. Mdagents: An adaptive collaboration of llms for medical decision-making. *Advances in Neural Information Processing Systems* 37 (2025), 79410–79452.
- [67] Kelvin J. L. Koa, Yunshan Ma, Ritchie Ng, and Tat-Seng Chua. 2024. Learning to generate explainable stock predictions using self-reflective large language models. In *Proceedings of the ACM Web Conference 2024*. 4304–4315.
- [68] Jakub Lála, Odhran O'Donoghue, Aleksandar Shtedritski, Sam Cox, Samuel G. Rodrigues, and Andrew D. White. 2023. Paperqa: Retrieval-augmented generative agent for scientific research. arXiv:2312.07559. Retrieved from <https://arxiv.org/abs/2312.07559>
- [69] Marc Lanctot, Edward Lockhart, Jean-Baptiste Lespiau, Vinicius Zambaldi, Satyaki Upadhyay, Julien Pérolat, Sriram Srinivasan, Finbarr Timbers, Karl Tuyls, Shayegan Omidshafiei, et al. 2019. OpenSpiel: A framework for reinforcement learning in games. arXiv:1908.09453. Retrieved from <https://arxiv.org/abs/1908.09453>
- [70] Dong Won Lee, Hae Park, Yoon Kim, Cynthia Breazeal, and Louis-Philippe Morency. 2024. Global reward to local rewards: Multimodal-guided decomposition for improving dialogue agents. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 15737–15762.
- [71] Binxu Li, Tiankai Yan, Yuanting Pan, Jie Luo, Ruiyang Ji, Jiayuan Ding, Zhe Xu, Shilong Liu, Haoyu Dong, Zihao Lin, et al. 2024. MMedAgent: Learning to use medical tools with multi-modal agent. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. 8745–8760.

- [72] Dawei Li, Bohan Jiang, Liangjie Huang, Alimohammad Beigi, Chengshuai Zhao, Zhen Tan, Amrita Bhattacharjee, Yuxuan Jiang, Canyu Chen, Tianhao Wu, et al. 2025. From generation to judgment: Opportunities and challenges of llm-as-a-judge. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. 2757–2791.
- [73] Dawei Li, Zhen Tan, Peijia Qian, Yifan Li, Kumar Satvik Chaudhary, Lijie Hu, and Jiayi Shen. 2024. Smoa: Improving multi-agent large language models with sparse mixture-of-agents. arXiv:2411.03284. Retrieved from <https://arxiv.org/abs/2411.03284>
- [74] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. 2023. CAMEL: Communicative agents for “mind” exploration of large language model society. In *Proceedings of the 37th International Conference on Neural Information Processing Systems*. 51991–52008.
- [75] Jinyang Li, Binyuan Hui, Ge Qu, Jiayi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, et al. 2023. Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls. *Advances in Neural Information Processing Systems* 36 (2023), 42330–42357.
- [76] Junkai Li, Yungwei Lai, Weitao Li, Jingyi Ren, Meng Zhang, Xinhui Kang, Siyu Wang, Peng Li, Ya-Qin Zhang, Weizhi Ma, et al. 2024. Agent hospital: A simulacrum of hospital with evolvable medical agents. arXiv:2405.02957. Retrieved from <https://arxiv.org/abs/2405.02957>
- [77] Ming Li, Yong Zhang, Zhitao Li, Jiuhai Chen, Lichang Chen, Ning Cheng, Jianzong Wang, Tianyi Zhou, and Jing Xiao. 2024. From quantity to quality: Boosting LLM performance with self-guided data selection for instruction tuning. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. 7595–7628.
- [78] Renhao Li, Minghuan Tan, Derek Wong, and Min Yang. 2024. CoEvol: Constructing better responses for instruction finetuning through multi-agent cooperation. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 4703–4721.
- [79] Xiaoxi Li, Jiajie Jin, Guanting Dong, Hongjin Qian, Yongkang Wu, Ji-Rong Wen, Yutao Zhu, and Zhicheng Dou. 2025. Webthinker: Empowering large reasoning models with deep research capability. arXiv:2504.21776. Retrieved from <https://arxiv.org/abs/2504.21776>
- [80] Xinyi Li, Sai Wang, Siqi Zeng, Yu Wu, and Yi Yang. 2024. A survey on LLM-based multi-agent systems: workflow, infrastructure, and challenges. *Viciniagearth* 1, 1 (2024), 9.
- [81] Yang Li, Yangyang Yu, Haohang Li, Zhi Chen, and Khaldoun Khashanah. 2023. Tradinggpt: Multi-agent system with layered memory and distinct characters for enhanced financial trading performance. arXiv:2309.03736. Retrieved from <https://arxiv.org/abs/2309.03736>
- [82] Zaijing Li, Yuquan Xie, Rui Shao, Gongwei Chen, Dongmei Jiang, and Liqiang Nie. 2024. Optimus-1: Hybrid multimodal memory empowered agents excel in long-horizon tasks. In *Proceedings of the Advances in Neural Information Processing Systems*, Vol. 37. 49881–49913.
- [83] Tian Liang, Zhiwei He, Wenxiang Jiao, Xing Wang, Yan Wang, Rui Wang, Yujiu Yang, Shuming Shi, and Zhaopeng Tu. 2024. Encouraging divergent thinking in large language models through multi-agent debate. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 17889–17904.
- [84] Xuechen Liang, Meiling Tao, Yinghui Xia, Tianyu Shi, Jun Wang, and JingSong Yang. 2024. Cmat: A multi-agent collaboration tuning framework for enhancing small language models. arXiv:2404.01663. Retrieved from <https://arxiv.org/abs/2404.01663>
- [85] Xuechen Liang, Meiling Tao, Yinghui Xia, Tianyu Shi, Jun Wang, and JingSong Yang. 2024. Self-evolving agents with reflective and memory-augmented abilities. arXiv:2409.00872. Retrieved from <https://arxiv.org/abs/2409.00872>
- [86] Bill Yuchen Lin, Yicheng Fu, Karina Yang, Faeze Brahman, Shiyu Huang, Chandra Bhagavatula, Prithviraj Ammanabrolu, Yejin Choi, and Xiang Ren. 2023. Swiftsage: A generative agent with fast and slow thinking for complex interactive tasks. *Advances in Neural Information Processing Systems* 36 (2023), 23813–23825.
- [87] Bill Yuchen Lin, Abhilasha Ravichander, Ximing Lu, Nouha Dziri, Melanie Sclar, Khyathi Chandu, Chandra Bhagavatula, and Yejin Choi. 2024. The unlocking spell on base LLMs: Rethinking alignment via in-context learning. In *Proceedings of the 12th International Conference on Learning Representations*.
- [88] Stephanie Lin, Jacob Hilton, and Owain Evans. 2022. TruthfulQA: Measuring how models mimic human falsehoods. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 3214–3252.
- [89] Evan Zheran Liu, Kelvin Guu, Panupong Pasupat, Tianlin Shi, and Percy Liang. 2018. Reinforcement learning on web interfaces using workflow-guided exploration. arXiv:1802.08802. Retrieved from <https://arxiv.org/abs/1802.08802>
- [90] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. 2023. Visual instruction tuning. *Advances in neural information processing systems* 36 (2023), 34892–34916.
- [91] Jerry Liu. 2022. *LlamaIndex*. DOI: <https://doi.org/10.5281/zenodo.1234>

- [92] Jie Liu, Pan Zhou, Yingjun Du, Ah-Hwee Tan, Cees G. M. Snoek, Jan-Jakob Sonke, and Efstratios Gavves. 2025. CaPo: Cooperative plan optimization for efficient embodied multi-agent cooperation. In *Proceedings of the 13th International Conference on Learning Representations*.
- [93] Sizhe Liu, Yizhou Lu, Siyu Chen, Xiyang Hu, Jieyu Zhao, Tianfan Fu, and Yue Zhao. 2024. Drugagent: Automating ai-aided drug discovery programming through llm multi-agent collaboration. arXiv:2411.15692. Retrieved from <https://arxiv.org/abs/2411.15692>
- [94] Xiao Liu, Kaixuan Ji, Yicheng Fu, Weng Tam, Zhengxiao Du, Zhilin Yang, and Jie Tang. 2022. P-tuning: Prompt tuning can be comparable to fine-tuning across scales and tasks. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*. 61–68.
- [95] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. 2024. AgentBench: Evaluating LLMs as agents. In *Proceedings of the 12th International Conference on Learning Representations*.
- [96] Yun-ting Liu, Jia-ming Yang, Liang Chen, Ting Guo, and Yu Jiang. 2020. Overview of reinforcement learning based on value and policy. In *Proceedings of the 2020 Chinese Control And Decision Conference*. IEEE, 598–603.
- [97] Zijun Liu, Yanzhe Zhang, Peng Li, Yang Liu, and Diyi Yang. 2024. A dynamic LLM-powered agent network for task-oriented agent collaboration. In *Proceedings of the 1st Conference on Language Modeling*.
- [98] Pan Lu, Swaroop Mishra, Tanglin Xia, Liang Qiu, Kai-Wei Chang, Song-Chun Zhu, Oyvind Tafjord, Peter Clark, and Ashwin Kalyan. 2022. Learn to explain: Multimodal reasoning via thought chains for science question answering. *Advances in Neural Information Processing Systems* 35 (2022), 2507–2521.
- [99] Andres M. Bran, Sam Cox, Oliver Schilter, Carlo Baldassari, Andrew D. White, and Philippe Schwaller. 2024. Augmenting large language models with chemistry tools. *Nature Machine Intelligence* 6, 5 (2024), 525–535.
- [100] Chang Ma, Junlei Zhang, Zhihao Zhu, Cheng Yang, Yujiu Yang, Yaohui Jin, Zhenzhong Lan, Lingpeng Kong, and Junxian He. 2024. AgentBoard: An analytical evaluation board of multi-turn LLM agents. In *Proceedings of the 38th Conference on Neural Information Processing Systems Datasets and Benchmarks Track*.
- [101] Hao Ma, Tianyi Hu, Zhiqiang Pu, Boyin Liu, Xiaolin Ai, Yanyan Liang, and Min Chen. 2024. Coevolving with the other you: Fine-tuning LLM with sequential cooperative multi-agent reinforcement learning. arXiv:2410.06101. Retrieved from <https://arxiv.org/abs/2410.06101>
- [102] Yubo Ma, Zhibin Gou, Junheng Hao, Ruochen Xu, Shuhang Wang, Liangming Pan, Yujiu Yang, Yixin Cao, and Aixun Sun. 2024. SciAgent: Tool-augmented language models for scientific reasoning. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 15701–15736.
- [103] Zhao Mandi, Shreeya Jain, and Shuran Song. 2024. Roco: Dialectic multi-robot collaboration with large language models. In *Proceedings of the 2024 IEEE International Conference on Robotics and Automation*. IEEE, 286–299.
- [104] Kartikeya Mangalam, Raiymbek Akshulakov, and Jitendra Malik. 2023. Egoschema: A diagnostic benchmark for very long-form video language understanding. *Advances in Neural Information Processing Systems* 36 (2023), 46212–46244.
- [105] Yuren Mao, Yuhang Ge, Yijiang Fan, Wenyi Xu, Yu Mi, Zhonghao Hu, and Yunjun Gao. 2025. A survey on lora of large language models. *Frontiers of Computer Science* 19, 7 (2025), 197605.
- [106] Grégoire Mialon, Clémentine Fourrier, Thomas Wolf, Yann LeCun, and Thomas Scialom. 2024. Gaia: A benchmark for general ai assistants. In *Proceedings of the 12th International Conference on Learning Representations*.
- [107] Shen-Yun Miao, Chao-Chun Liang, and Keh-Yih Su. 2020. A diverse corpus for evaluating and developing english math word problem solvers. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. 975–984.
- [108] Mahmoud Mohammadi, Yipeng Li, Jane Lo, and Wendy Yip. 2025. Evaluation and benchmarking of llm agents: A survey. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*. 6129–6139.
- [109] Ofir Nachum, Mohammad Norouzi, Kelvin Xu, and Dale Schuurmans. 2017. Bridging the gap between value and policy based reinforcement learning. *Advances in Neural Information Processing Systems* 30 (2017), 2772–2782.
- [110] King Han Naman Jain, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. 2024. Livecodebench: Holistic and contamination free evaluation of large language models for code. arXiv:2403.07974. Retrieved from <https://arxiv.org/abs/2403.07974>
- [111] Alexander Novikov, Ngăn Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, et al. 2025. AlphaEvolve: A coding agent for scientific and algorithmic discovery. arXiv:2506.13131. Retrieved from <https://arxiv.org/abs/2506.13131>
- [112] OpenAI. 2024. GPT-4 Technical Report. arXiv:2303.08774. Retrieved from <https://arxiv.org/abs/2303.08774>
- [113] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems* 35 (2022), 27730–27744.
- [114] Jing-Cheng Pang, Si-Hang Yang, Kaiyuan Li, Jiaji Zhang, Xiong-Hui Chen, Nan Tang, and Yang Yu. 2024. KALM: Knowledgeable agents by offline reinforcement learning from large language model rollouts. In *Proceedings of the Advances in Neural Information Processing Systems*. 126620–126652.

- [115] Venkatesh Balavadhani Parthasarathy, Ahtsham Zafar, Aafaq Khan, and Arsalan Shahid. 2024. The ultimate guide to fine-tuning llms from basics to breakthroughs: An exhaustive review of technologies, research, best practices, applied research challenges and opportunities. arXiv:2408.13296. Retrieved from <https://arxiv.org/abs/2408.13296>
- [116] Arkil Patel, Satwik Bhattamishra, and Navin Goyal. 2021. Are NLP models really able to solve simple math word problems?. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. 2080–2094.
- [117] Jan Peters and Stefan Schaal. 2006. Policy gradient methods for robotics. In *Proceedings of the 2006 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, 2219–2225.
- [118] Pranav Putta, Edmund Mills, Naman Garg, Sumeet Motwani, Chelsea Finn, Divyansh Garg, and Rafael Rafailov. 2024. Agent q: Advanced reasoning and learning for autonomous ai agents. arXiv:2408.07199. Retrieved from <https://arxiv.org/abs/2408.07199>
- [119] Zehan Qi, Xiao Liu, Iat Long Long, Hanyu Lai, Xueqiao Sun, Wenyi Zhao, Yu Yang, Xinyue Yang, Jiadai Sun, Shuntian Yao, et al. 2024. WebRL: Training LLM web agents via self-evolving online curriculum reinforcement learning. arXiv:2411.02337. Retrieved from <https://arxiv.org/abs/2411.02337>
- [120] Chen Qian, Yufan Dang, Jiahao Li, Wei Liu, Zihao Xie, YiFei Wang, Weize Chen, Cheng Yang, Xin Cong, Xiaoyin Che, Zhiyuan Liu, and Maosong Sun. 2024. Experiential co-learning of software-developing agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 5628–5640.
- [121] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, et al. 2024. ChatDev: Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 15174–15186.
- [122] Chen Qian, Zihao Xie, YiFei Wang, Wei Liu, Yufan Dang, Zhuoyun Du, Weize Chen, Cheng Yang, Zhiyuan Liu, and Maosong Sun. 2024. Scaling large-language-model-based multi-agent collaboration. arXiv:2406.07155. Retrieved from <https://arxiv.org/abs/2406.07155>
- [123] Shuofei Qiao, Ningyu Zhang, Runnan Fang, Yuqi Zhu, Xiang Chen, Shumin Deng, Yong Jiang, Pengjun Xie, Fei Huang, and Huajun Chen. 2024. Agent planning with world knowledge model. *Advances in Neural Information Processing Systems* 37 (2024), 114843–114871.
- [124] Shuofei Qiao, Ningyu Zhang, Runnan Fang, Yujie Luo, Wangchunshu Zhou, Yuchen Jiang, Chengfei Lv, and Huajun Chen. 2024. AutoAct: Automatic agent learning from scratch for QA via self-planning. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 3003–3021.
- [125] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. 2023. Toolllm: Facilitating large language models to master 16000+ real-world apis. arXiv:2307.16789. Retrieved from <https://arxiv.org/abs/2307.16789>
- [126] Yuxiao Qu, Tianjun Zhang, Naman Garg, and Aviral Kumar. 2024. Recursive introspection: Teaching language model agents how to self-improve. *Advances in Neural Information Processing Systems* 37 (2024), 55249–55285.
- [127] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D. Manning, Stefano Ermon, and Chelsea Finn. 2023. Direct preference optimization: Your language model is secretly a reward model. *Advances in Neural Information Processing Systems* 36 (2023), 53728–53741.
- [128] Matthew Renze and Erhan Guven. 2024. Self-reflection in LLM agents: Effects on problem-solving performance. arXiv:2405.06682. Retrieved from <https://arxiv.org/abs/2405.06682>
- [129] Yusuf Roohani, Andrew Lee, Qian Huang, Jian Vora, Zachary Steinhart, Kexin Huang, Alexander Marson, Percy Liang, and Jure Leskovec. 2024. Biodiscoveryagent: An ai agent for designing genetic perturbation experiments. arXiv:2405.17631. Retrieved from <https://arxiv.org/abs/2405.17631>
- [130] Jingqing Ruan, Yihong Chen, Bin Zhang, Zhiwei Xu, Tianpeng Bao, Guoqing Du, Shiwei Shi, Hangyu Mao, Ziyue Li, Xingyu Zeng, et al. 2023. TPTU: Large language model-based AI agents for task planning and tool usage. arXiv:2308.03427. Retrieved from <https://arxiv.org/abs/2308.03427>
- [131] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. arXiv:1707.06347. Retrieved from <https://arxiv.org/abs/1707.06347>
- [132] Dustin Schwenk, Apoorv Khandelwal, Christopher Clark, Kenneth Marino, and Roozbeh Mottaghi. 2022. A-okvqa: A benchmark for visual question answering using world knowledge. In *Proceedings of the European Conference on Computer Vision*. Springer, 146–162.
- [133] Jinxin Shi, Zongsheng Cao, Runmin Ma, Yusong Hu, Jie Zhou, Xin Li, Lei Bai, Liang He, and Bo Zhang. 2025. DualResearch: Entropy-gated dual-graph retrieval for answer reconstruction. arXiv:2510.08959. Retrieved from <https://arxiv.org/abs/2510.08959>
- [134] Wentao Shi, Zichun Yu, Fuli Feng, Xiangnan He, and Chenyan Xiong. 2025. Efficient multi-agent system training with data influence-oriented tree search. arXiv:2502.00955. Retrieved from <https://arxiv.org/abs/2502.00955>

- [135] Wentao Shi, Mengqi Yuan, Junkang Wu, Qifan Wang, and Fuli Feng. 2024. Direct multi-turn preference optimization for language agents. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 2312–2324.
- [136] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems* 36 (2023), 8634–8652.
- [137] Mohit Shridhar, Jesse Thomason, Daniel Gordon, Yonatan Bisk, Winson Han, Roozbeh Mottaghi, Luke Zettlemoyer, and Dieter Fox. 2020. Alfred: A benchmark for interpreting grounded instructions for everyday tasks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 10740–10749.
- [138] Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. 2021. ALFWorld: Aligning text and embodied environments for interactive learning. In *Proceedings of the International Conference on Learning Representations*.
- [139] Karan Singhal, Shekoofeh Azizi, Tao Tu, S. Sara Mahdavi, Jason Wei, Hyung Won Chung, Nathan Scales, Ajay Tanwani, Heather Cole-Lewis, Stephen Pfohl, et al. 2023. Large language models encode clinical knowledge. *Nature* 620, 7972 (2023), 172–180.
- [140] Yifan Song, Weimin Xiong, Xiutian Zhao, Dawei Zhu, Wenhao Wu, Ke Wang, Cheng Li, Wei Peng, and Sujian Li. 2024. AgentBank: Towards generalized LLM agents via fine-tuning on 50000+ interaction trajectories. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. 2124–2141.
- [141] Yifan Song, Da Yin, Xiang Yue, Jie Huang, Sujian Li, and Bill Yuchen Lin. 2024. Trial and error: Exploration-based trajectory optimization of LLM agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 7584–7600.
- [142] Haoyang Su, Renqi Chen, Shixiang Tang, Zhenfei Yin, Xinzhe Zheng, Jinzhe Li, Biqing Qi, Qi Wu, Hui Li, Wanli Ouyang, Philip Torr, Bowen Zhou, and Nanqing Dong. 2024. Many heads are better than one: Improved scientific idea generation by a LLM-based multi-agent system. arXiv:2410.09403. Retrieved from <https://arxiv.org/abs/2410.09403>
- [143] Vighnesh Subramaniam, Yilun Du, Joshua B. Tenenbaum, Antonio Torralba, Shuang Li, and Igor Mordatch. 2025. Multiagent finetuning: Self improvement with diverse reasoning chains. arXiv:2501.05707. Retrieved from <https://arxiv.org/abs/2501.05707>
- [144] Hao Sun, Jiayi Wu, Hengyi Cai, Xiaochi Wei, Yue Feng, Bo Wang, Shuaiqiang Wang, Yan Zhang, and Dawei Yin. 2024. AdaSwitch: Adaptive switching between small and large agents for effective cloud-local collaborative learning. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 8052–8062.
- [145] Richard S. Sutton, David McAllester, Satinder Singh, and Yishay Mansour. 1999. Policy gradient methods for reinforcement learning with function approximation. *Advances in Neural Information Processing Systems* 12 (1999), 1057–1063.
- [146] Zhengwei Tao, Ting-En Lin, Xiancai Chen, Hangyu Li, Yuchuan Wu, Yongbin Li, Zhi Jin, Fei Huang, Dacheng Tao, and Jingren Zhou. 2024. A survey on self-evolution of large language models. arXiv:2404.14387. Retrieved from <https://arxiv.org/abs/2404.14387>
- [147] InternAgent Team, Bo Zhang, Shiyang Feng, Xiangchao Yan, Jiakang Yuan, Runmin Ma, Yusong Hu, Zhiyin Yu, Xiaohan He, Songtao Huang, et al. 2025. InternAgent: When agent becomes the scientist—building closed-loop system from hypothesis to verification. *arXiv e-prints* (2025), arXiv–2505.
- [148] Microsoft Team. 2025. Semantic Kernel. Retrieved December 2, 2025 from <https://github.com/microsoft/semantic-kernel>
- [149] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2022. MuSiQue: Multihop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics* 10 (2022), 539–554.
- [150] Luong Trung, Xinbo Zhang, Zhanming Jie, Peng Sun, Xiaoran Jin, and Hang Li. 2024. Reft: Reasoning with reinforced fine-tuning. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 7601–7614.
- [151] Dennis Ulmer, Elman Mansimov, Kaixiang Lin, Justin Sun, Xibin Gao, and Yi Zhang. 2024. Bootstrapping llm-based task-oriented dialogue agents via self-talk. arXiv:2401.05033. Retrieved from <https://arxiv.org/abs/2401.05033>
- [152] Boshi Wang, Hao Fang, Jason Eisner, Benjamin Van Durme, and Yu Su. 2024. LLMs in the imaginary: Tool learning through simulated trial and error. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 10583–10604.
- [153] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023. Voyager: An open-ended embodied agent with large language models. arXiv:2305.16291. Retrieved from <https://arxiv.org/abs/2305.16291>
- [154] Hanlin Wang, Chak Tou Leong, Jian Wang, and Wenjie Li. 2024. E2CL: Exploration-based error correction learning for embodied agents. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. 7626–7639.
- [155] Hanlin Wang, Jian Wang, Chak Tou Leong, and Wenjie Li. 2025. Steca: Step-level trajectory calibration for llm agent learning. arXiv:2502.14276. Retrieved from <https://arxiv.org/abs/2502.14276>

- [156] Pei Wang, Yanan Wu, Zekun Wang, Jiaheng Liu, Xiaoshuai Song, Zhongyuan Peng, Ken Deng, Chenchen Zhang, Jiakai Wang, Junran Peng, et al. 2024. Mtu-bench: A multi-granularity tool-use benchmark for large language models. arXiv:2410.11710. Retrieved from <https://arxiv.org/abs/2410.11710>
- [157] Ruoyao Wang, Peter Jansen, Marc-Alexandre Côté, and Prithviraj Ammanabrolu. 2022. ScienceWorld: Is your agent smarter than a 5th grader?. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*. 11279–11298.
- [158] Renxi Wang, Haonan Li, Xudong Han, Yixuan Zhang, and Timothy Baldwin. 2024. Learning from failure: Integrating negative examples when fine-tuning large language models as agents. arXiv:2402.11651. Retrieved from <https://arxiv.org/abs/2402.11651>
- [159] Xingyao Wang, Zihan Wang, Jiateng Liu, Yangyi Chen, Lifan Yuan, Hao Peng, and Heng Ji. 2024. MINT: Evaluating LLMs in multi-turn interaction with tools and language feedback. In *Proceedings of the 12th International Conference on Learning Representations*. 1–35.
- [160] Zheng Wang, Zhongyang Li, Zeren Jiang, Dandan Tu, and Wei Shi. 2024. Crafting personalized agents through retrieval-augmented generation on editable memory graphs. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 4891–4906.
- [161] Christopher J. C. H. Watkins and Peter Dayan. 1992. Q-learning. *Machine Learning* 8, 3–4 (1992), 279–292.
- [162] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V. Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems* 35 (2022), 24824–24837.
- [163] Muning Wen, Ziyu Wan, Weinan Zhang, Jun Wang, and Ying Wen. 2024. Reinforcing language agents via policy optimization with action decomposition. arXiv:2405.15821. Retrieved from <https://arxiv.org/abs/2405.15821>
- [164] Cheng-Kuang Wu, Zhi Rui Tam, Chieh-Yen Lin, Yun-Nung Chen, and Hung-yi Lee. 2024. StreamBench: Towards benchmarking continuous improvement of language agents. In *Proceedings of the 38th Conference on Neural Information Processing Systems Datasets and Benchmarks Track*.
- [165] Jialong Wu, Shaofeng Yin, Ningya Feng, Xu He, Dong Li, Jianye Hao, and Mingsheng Long. 2025. ivideopt: Interactive videopts are scalable world models. *Advances in Neural Information Processing Systems* 37 (2025), 68082–68119.
- [166] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. 2023. Autogen: Enabling next-gen llm applications via multi-agent conversation. arXiv:2308.08155. Retrieved from <https://arxiv.org/abs/2308.08155>
- [167] Shirley Wu, Shiyu Zhao, Qian Huang, Kexin Huang, Michihiro Yasunaga, Kaidi Cao, Vassilis Ioannidis, Karthik Subbian, Jure Leskovec, and James Y. Zou. 2025. AvaTaR: Optimizing LLM agents for tool usage via contrastive reasoning. *Advances in Neural Information Processing Systems* 37 (2025), 25981–26010.
- [168] Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, et al. 2025. The rise and potential of large language model based agents: A survey. *Science China Information Sciences* 68, 2 (2025), 121101.
- [169] Zhiheng Xi, Yiwen Ding, Wenxiang Chen, Boyang Hong, Honglin Guo, Junzhe Wang, Dingwen Yang, Chenyang Liao, Xin Guo, Wei He, et al. 2024. AgentGym: Evolving large language model-based agents across diverse environments. arXiv:2406.04151. Retrieved from <https://arxiv.org/abs/2406.04151>
- [170] Zhiheng Xi, Jixuan Huang, Chenyang Liao, Baodai Huang, Honglin Guo, Jiaqi Liu, Rui Zheng, Junjie Ye, Jiazheng Zhang, Wenxiang Chen, et al. 2025. Agentgym-rl: Training llm agents for long-horizon decision making through multi-turn reinforcement learning. arXiv:2509.08755. Retrieved from <https://arxiv.org/abs/2509.08755>
- [171] Yufei Xiang, Yiqun Shen, Yeqin Zhang, and Nguyen Cam-Tu. 2024. Retrospect: Language agent meets offline reinforcement learning critic. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 4650–4666.
- [172] Junbin Xiao, Xindi Shang, Angela Yao, and Tat-Seng Chua. 2021. Next-qa: Next phase of question-answering to explaining temporal actions. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 9777–9786.
- [173] Yihang Xiao, Jinyi Liu, Yan Zheng, Xiaohan Xie, Jianye Hao, Mingzhi Li, Ruitao Wang, Fei Ni, Yuxiao Li, Jintian Luo, et al. 2024. Cellagent: An llm-driven multi-agent framework for automated single-cell data analysis. arXiv:2407.09811. Retrieved from <https://arxiv.org/abs/2407.09811>
- [174] Yijia Xiao, Edward Sun, Di Luo, and Wei Wang. 2024. TradingAgents: Multi-agents LLM financial trading framework. arXiv:2412.20138. Retrieved from <https://arxiv.org/abs/2412.20138>
- [175] Ziyang Xiao, Dongxiang Zhang, Yangjun Wu, Lilin Xu, Yuan Jessica Wang, Xiongwei Han, Xiaojin Fu, Tao Zhong, Jia Zeng, Mingli Song, et al. 2023. Chain-of-experts: When llms meet complex operations research problems. In *Proceedings of the 12th International Conference on Learning Representations*.
- [176] Weimin Xiong, Yifan Song, Qingxiu Dong, Bingchan Zhao, Feifan Song, Xun Wang, and Sujian Li. 2025. MPO: Boosting LLM agents with meta plan optimization. arXiv:2503.02682. Retrieved from <https://arxiv.org/abs/2503.02682>

- [177] Weimin Xiong, Yifan Song, Xiutian Zhao, Wenhao Wu, Xun Wang, Ke Wang, Cheng Li, Wei Peng, and Sujian Li. 2024. Watch every step! LLM agent learning via iterative step-level process refinement. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 1556–1572.
- [178] Fangzhi Xu, Qiushi Sun, Kanzhi Cheng, Jun Liu, Yu Qiao, and Zhiyong Wu. 2024. Interactive evolution: A neural-symbolic self-training framework for large language models. arXiv:2406.11736. Retrieved from <https://arxiv.org/abs/2406.11736>
- [179] Renjun Xu and Jingwen Peng. 2025. A comprehensive survey of deep research: Systems, methodologies, and applications. arXiv:2506.12594. Retrieved from <https://arxiv.org/abs/2506.12594>
- [180] Tianyang Xu, Shujin Wu, Shizhe Diao, Xiaozhe Liu, Xingyao Wang, Yangyi Chen, and Jing Gao. 2024. SaySelf: Teaching LLMs to express confidence with self-reflective rationales. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 5985–5998.
- [181] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. 2024. Large language models as optimizers. In *Proceedings of the 12th International Conference on Learning Representations, ICLR 2024*.
- [182] Hongyang Yang, Boyu Zhang, Neng Wang, Cheng Guo, Xiaoli Zhang, Likun Lin, Junlin Wang, Tianyu Zhou, Mao Guan, Runjia Zhang, et al. 2024. FinRobot: An open-source AI agent platform for financial applications using large language models. arXiv:2405.14767. Retrieved from <https://arxiv.org/abs/2405.14767>
- [183] John Yang, Carlos E. Jimenez, Alex L. Zhang, Kilian Lieret, Joyce Yang, Xindi Wu, Ori Press, Niklas Muennighoff, Gabriel Synnaeve, Karthik R. Narasimhan, et al. 2025. SWE-bench multimodal: Do AI systems generalize to visual software domains?. In *Proceedings of the 13th International Conference on Learning Representations*.
- [184] John Yang, Akshara Prabhakar, Karthik Narasimhan, and Shunyu Yao. 2023. Intercode: Standardizing and benchmarking interactive coding with execution feedback. *Advances in Neural Information Processing Systems* 36 (2023), 23826–23854.
- [185] Yijun Yang, Tianyi Zhou, Kanxue Li, Dapeng Tao, Lusong Li, Li Shen, Xiaodong He, Jing Jiang, and Yuhui Shi. 2024. Embodied multi-modal agent trained by an llm from a parallel textworld. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 26275–26285.
- [186] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. 2369–2380.
- [187] Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. 2022. Webshop: Towards scalable real-world web interaction with grounded language agents. *Advances in Neural Information Processing Systems* 35 (2022), 20744–20757.
- [188] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing reasoning and acting in language models. In *Proceedings of the International Conference on Learning Representations*.
- [189] Weiran Yao, Shelby Heinecke, Juan Carlos Niebles, Zhiwei Liu, Yihao Feng, Le Xue, Rithesh Murthy, Zeyuan Chen, Jianguo Zhang, Devansh Arpit, et al. 2023. Retroformer: Retrospective large language agents with policy gradient optimization. arXiv:2308.02151. Retrieved from <https://arxiv.org/abs/2308.02151>
- [190] Geyan Ye, Xibao Cai, Houtim Lai, Xing Wang, Junhong Huang, Longyue Wang, Wei Liu, and Xiangxiang Zeng. 2025. Drugassist: A large language model for molecule optimization. *Briefings in Bioinformatics* 26, 1 (2025), bbae693.
- [191] Da Yin, Faeze Brahman, Abhilasha Ravichander, Khyathi Chandu, Kai-Wei Chang, Yejin Choi, and Bill Yuchen Lin. 2024. Agent lumos: Unified and modular training for open-source language agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 12380–12403.
- [192] Yangyang Yu, Haohang Li, Zhi Chen, Yuechen Jiang, Yang Li, Denghui Zhang, Rong Liu, Jordan W. Suchow, and Khaldoun Khashanah. 2024. FinMem: A performance-enhanced LLM trading agent with layered memory and character design. In *Proceedings of the AAAI Symposium Series*. 595–597.
- [193] Yangyang Yu, Zhiyuan Yao, Haohang Li, Zhiyang Deng, Yuechen Jiang, Yupeng Cao, Zhi Chen, Jordan Suchow, Zhenyu Cui, Rong Liu, et al. 2025. Fincon: A synthesized llm multi-agent system with conceptual verbal reinforcement for enhanced financial decision making. *Advances in Neural Information Processing Systems* 37 (2025), 137010–137045.
- [194] Ling Yue, Sixue Xing, Jintai Chen, and Tianfan Fu. 2024. Clinicalagent: Clinical trial multi-agent system with large language model-based reasoning. In *Proceedings of the 15th ACM International Conference on Bioinformatics, Computational Biology and Health Informatics*. 1–10.
- [195] Shengbin Yue, Siyuan Wang, Wei Chen, Xuanjing Huang, and Zhongyu Wei. 2024. Synergistic multi-agent framework with trajectory learning for knowledge-intensive tasks. arXiv:2407.09893. Retrieved from <https://arxiv.org/abs/2407.09893>
- [196] Kamer Ali Yuksel and Hassan Sawaf. 2024. A Multi-AI Agent System for Autonomous Optimization of Agentic AI Solutions via Iterative Refinement and LLM-Driven Feedback Loops. arXiv:2412.17149. Retrieved from <https://arxiv.org/abs/2412.17149>

- [197] Aohan Zeng, Mingdao Liu, Rui Lu, Bowen Wang, Xiao Liu, Yuxiao Dong, and Jie Tang. 2023. Agenttuning: Enabling generalized agent abilities for llms. arXiv:2310.12823. Retrieved from <https://arxiv.org/abs/2310.12823>
- [198] Yifan Zeng, Yiran Wu, Xiao Zhang, Huazheng Wang, and Qingyun Wu. 2024. Autodefense: Multi-agent llm defense against jailbreak attacks. arXiv:2403.04783. Retrieved from <https://arxiv.org/abs/2403.04783>
- [199] Daochen Zha, Kwei-Heng Lai, Yuanpu Cao, Songyi Huang, Ruzhe Wei, Junyu Guo, and Xia Hu. 2019. Rlcard: A toolkit for reinforcement learning in card games. arXiv:1910.04376. Retrieved from <https://arxiv.org/abs/1910.04376>
- [200] Simon Zhai, Hao Bai, Zipeng Lin, Jiayi Pan, Peter Tong, Yifei Zhou, Alane Suhr, Saining Xie, Yann LeCun, Yi Ma, et al. 2024. Fine-tuning large vision-language models as decision-making agents via reinforcement learning. *Advances in Neural Information Processing Systems* 37 (2024), 110935–110971.
- [201] Hongxin Zhang, Weihua Du, Jiaming Shan, Qinhong Zhou, Yilun Du, Joshua B. Tenenbaum, Tianmin Shu, and Chuang Gan. 2024. Building cooperative embodied agents modularly with large language models. In *Proceedings of the International Conference on Learning Representations*.
- [202] Jianguo Zhang, Tian Lan, Rithesh Murthy, Zhiwei Liu, Weiran Yao, Ming Zhu, Juntao Tan, Thai Hoang, Zuxin Liu, Liangwei Yang, et al. 2024. Agentohana: Design unified data and training pipeline for effective agent learning. arXiv:2402.15506. Retrieved from <https://arxiv.org/abs/2402.15506>
- [203] Kaiqing Zhang, Zhuoran Yang, and Tamer Başar. 2021. Multi-agent reinforcement learning: A selective overview of theories and algorithms. *Handbook of Reinforcement Learning and Control* (2021), 321–384.
- [204] Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, et al. 2025. Agentic context engineering: Evolving contexts for self-improving language models. arXiv:2510.04618. Retrieved from <https://arxiv.org/abs/2510.04618>
- [205] Shengyu Zhang, Linfeng Dong, Xiaoya Li, Sen Zhang, Xiaofei Sun, Shuhe Wang, Jiwei Li, Runyi Hu, Tianwei Zhang, Fei Wu, et al. 2023. Instruction tuning for large language models: A survey. arXiv:2308.10792. Retrieved from <https://arxiv.org/abs/2308.10792>
- [206] Shaokun Zhang, Jieyu Zhang, Jiale Liu, Linxin Song, Chi Wang, Ranjay Krishna, and Qingyun Wu. 2024. Offline training of language model agents with functions as learnable weights. *Proceedings of Machine Learning Research* 235 (2024), 60315–60335.
- [207] Wenqi Zhang, Ke Tang, Hai Wu, Mengna Wang, Yongliang Shen, Guiyang Hou, Zeqi Tan, Peng Li, Yueting Zhuang, and Weiming Lu. 2024. Agent-Pro: Learning to evolve via policy-level reflection and optimization. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 5348–5375.
- [208] Wentao Zhang, Lingxuan Zhao, Haochong Xia, Shuo Sun, Jiaze Sun, Molei Qin, Xinyi Li, Yuqing Zhao, Yilei Zhao, Xinyu Cai, et al. 2024. A multimodal foundation agent for financial trading: Tool-augmented, diversified, and generalist. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 4314–4325.
- [209] Yiming Zhang, Zheng Chang, Wentao Cai, MengXing Ren, Kang Yuan, Yining Sun, and Zenghui Ding. 2025. IIMedGPT: Promoting large language model capabilities of medical tasks by efficient human preference alignment. arXiv:2501.02869. Retrieved from <https://arxiv.org/abs/2501.02869>
- [210] Zeyu Zhang, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Quanyu Dai, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. 2024. A survey on the memory mechanism of large language model based agents. arXiv:2404.13501. Retrieved from <https://arxiv.org/abs/2404.13501>
- [211] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. Expel: LLM agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*. 19632–19642.
- [212] Qi Zhao, Haotian Fu, Chen Sun, and George Konidaris. 2024. EPO: Hierarchical LLM agents with environment preference optimization. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 6401–6415.
- [213] Chuanyang Zheng, Haiming Wang, Enze Xie, Zhengying Liu, Jiankai Sun, Huajian Xin, Jianhao Shen, Zhenguo Li, and Yu Li. 2024. Lyra: Orchestrating dual correction in automated theorem proving. *Transactions on Machine Learning Research* (2024).
- [214] Yuxiang Zheng, Dayuan Fu, Xiangkun Hu, Xiaojie Cai, Lyumanshan Ye, Pengrui Lu, and Pengfei Liu. 2025. Deepresearcher: Scaling deep research via reinforcement learning in real-world environments. arXiv:2504.03160. Retrieved from <https://arxiv.org/abs/2504.03160>
- [215] Hang Zhou, Yehui Tang, Haochen Qin, Yujie Yang, Renren Jin, Deyi Xiong, Kai Han, and Yunhe Wang. 2024. Star-agents: Automatic data optimization with LLM agents for instruction tuning. *Advances in Neural Information Processing Systems* 37 (2024), 4575–4597.
- [216] Han Zhou, Xingchen Wan, Ruoxi Sun, Hamid Palangi, Shariq Iqbal, Ivan Vulić, Anna Korhonen, and Sercan Ö Arık. 2025. Multi-agent design: Optimizing agents with better prompts and topologies. arXiv:2502.02533. Retrieved from <https://arxiv.org/abs/2502.02533>

- [217] Qin hao Zhou, Zihan Zhang, Xiang Xiang, Ke Wang, Yuchuan Wu, and Yongbin Li. 2024. Enhancing the general agent capabilities of low-paramter LLMs through tuning and multi-branch reasoning. In *Findings of the Association for Computational Linguistics: NAACL 2024*. 2922–2931.
- [218] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, et al. 2023. Webarena: A realistic web environment for building autonomous agents. arXiv:2307.13854. Retrieved from <https://arxiv.org/abs/2307.13854>
- [219] Wangchunshu Zhou, Yixin Ou, Shengwei Ding, Long Li, Jialong Wu, Tiannan Wang, Jiamin Chen, Shuai Wang, Xiaohua Xu, Ningyu Zhang, et al. 2024. Symbolic learning enables self-evolving agents. arXiv:2406.18532. Retrieved from <https://arxiv.org/abs/2406.18532>
- [220] Junda Zhu, Lingyong Yan, Haibo Shi, Dawei Yin, and Lei Sha. 2024. ATM: Adversarial tuning multi-agent system makes a robust retrieval-augmented generator. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 10902–10919.
- [221] Kaiwen Zuo, Yirui Jiang, Fan Mo, and Pietro Lio. 2024. KG4Diagnosis: A hierarchical multi-agent LLM framework with knowledge graph enhancement for medical diagnosis. arXiv:2412.16833. Retrieved from <https://arxiv.org/abs/2412.16833>

Received 22 April 2025; revised 5 December 2025; accepted 8 January 2026